Improving and Optimizing NeCT for NeCTv2

Master's Thesis

Department of Computer Science / Department of Physics
Norwegian University of Science and Technology (NTNU)

Kristian Gautefall Carlenius

Supervisor: *Ole Jakob Mengshoel & Dag Werner Breiby*

April 7, 2026

Abstract

[TODO: Writing the abstract last, summarise motivation, methods, key results, and conclusions aiming for 250 words.]

Acknowledgements

[TODO: Acknowledgements.]

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Understanding processes that unfold inside materials is one of the persistent challenges in both natural sciences and engineering. In geosciences, questions about how fluids move through rock pores, how minerals dissolve and precipitate under changing chemical conditions, and how rock deforms under mechanical stress all require direct observation of the three-dimensional internal structure as it changes over time. X-ray computed tomography has become the principal tool for this kind of observation, enabling non-destructive imaging of an internal structure at resolutions down to the micrometre scale. What remains difficult is time.

A conventional micro-CT scan of a rock sample takes on the order of one hour to acquire a single three-dimensional image. For a process such as brine displacing oil in a pore network, or a salt grain dissolving as a fluid front advances, the relevant events may unfold on timescales of seconds. The gap between what is physically happening and what the direct imaging can resolve in time is many orders of magnitude. Bridging this gap requires smarter acquisition schemes or reconstruction algorithms that extract more information from fewer measurements. Ideally some combination of both.

1.1 Background and Motivation

The standard approach to four-dimensional CT, denoted 4D-CT for the three spatial dimensions plus one dimention of time, is to acquire a sequence of independent full three-dimensional scans and reconstruct each independently. The temporal resolution of this approach is bounded below by the time required to complete one full 360ř rotation of the sample, which for laboratory instruments is governed by the photon flux of the X-ray source, the required signal-to-noise ratio per projection, and the number of angular positions needed for an artefact-free reconstruction. None of these constraints are easily relaxed without compromising spatial resolution or introducing reconstruction artefacts.

Reducing the number of projections per time step allows more time steps to be packed into a fixed total acquisition time, but with fewer projections the harder a accurate reconstruction becomes. Classical reconstruction algorithms such as the Feldkamp–Davis–Kress (FDK) algorithm and iterative methods with total variation regularisation handle sparse-view data poorly. FDK produces severe streak artefacts when the Shannon–Nyquist sampling criterion is not met, while total variation regularisation suppresses streaks at the cost of contrast reduction and loss of fine features. Neither algorithm uses information from other time steps to improve a given frame, so the temporal and spatial resolution trade-off is handled entirely within each independent reconstruction.

Machine learning approaches have been applied to sparse-view CT with considerable success on static images. Convolutional networks trained to map degraded FDK reconstructions to cleaner images can suppress artefacts effectively, but require large paired training datasets assembled in advance. More fundamentally, they have no internal model of the measurement physics, which means they are prone to generating plausible-looking but physically incorrect features when the test sample differs from the training distribution. For scientific imaging, where the correctness of individual pore-level features may carry direct physical consequences, this is a significant liability.

Implicit neural representations (INRs) offer a different approach. Rather than learning a mapping from degraded to clean images on a population of examples, an INR is fitted to the measurements of a single specific sample using the known physics of the CT measurement process as a differentiable forward model. The network receives no prior training data; instead it is optimised from scratch for each scan, constrained to produce predictions that are consistent with the measured projections under Beer–Lambert's law. This instance-specific approach eliminates the hallucination problem and makes the reconstruction inherently physics-informed.

Neural Computed Tomography (NeCT), introduced by Friis et al. [2025], is the first algorithm to extend this INR-based approach to fully four-dimensional CT. Rather than reconstructing each time step independently, NeCT represents the entire dynamic experiment as a single continuous function $\mu(\mathbf{r}, t)$ using a hash-encoded neural network, trained jointly on all projections collected over the full duration of the scan. This holistic treatment allows the reconstruction to exploit angular information from all time steps simultaneously, enabling temporal resolutions approaching the time between individual projections with standard laboratory instruments.

The NeCT paper established the viability of the approach through experiments on both synthetic dynamic datasets and a real imbibition experiment on Bentheimer sandstone, demonstrating that individual pore-filling events and salt dissolution processes on timescales of 10 to 30 seconds could be directly observed with laboratory

equipment. However, the paper treated the encoder configuration as a fixed design choice and evaluated the model under a single maximally large configuration, measuring reconstruction quality as the primary outcome without considering computational cost. The configuration adopted requires GPU memory in excess of 60 GB, restricting training to the most expensive research-grade hardware available, and training times on the order of tens of hours further limit the practical applicability of the method.

This thesis builds directly on NeCT with the aim of reducing the computational cost and increase functionality. It investigates three questions that the original paper left open: whether the encoder configuration can be reduced in size without a proportional loss of reconstruction quality, whether alternative encoder architectures offer a better trade-off between computational cost and accuracy, and whether the NeCT forward model can be extended to handle continuously acquired data in which the sample rotates without stopping between exposures.

1.2 Research Objectives

The work presented in this thesis is organised around three concrete research objectives, each corresponding to one experimental chapter.

The first objective is to characterise the relationship between encoder capacity and reconstruction quality in the NeCT QuadCubes architecture. The NeCT paper used the largest feasible configuration and trained for as long as possible, treating quality as the only optimisation target. This thesis instead asks whether there is a meaningful quality-versus-cost Pareto frontier, configurations that are substantially cheaper in memory and training time while achieving competitive reconstruction quality. Understanding this frontier is a prerequisite for using NeCT on hardware that is more widely accessible than an A100 or H100 GPU, and for workflows where reconstructions must be produced quickly enough to guide ongoing experiments.

The second objective is to evaluate alternative encoder architectures for the 4D hash encoding problem. The QuadCubes design uses four 3D multiresolution hash encoders covering all four possible 3-element subsets of the space-time coordinates. This is one point in a larger design space bounded by a single 4D encoder on one side, which suffers from a severe hash collision problem, and six 2D encoders covering all coordinate pairs on the other, which decouple the coordinates too strongly and require the MLP to perform a heavier integration task. This thesis evaluates a SingleCube (one 4D encoder), a SexCubes (six 2D encoders), and a CombinedCubes design (one 3D spatial encoder combined with three 2D space-time encoders), comparing all of them to QuadCubes on the same dataset. Additionally, a TriCubes architecture using three 2D spatial encoders covering the (x, y), (x, z), and (y, z) planes is evaluated as an alternative to the single 3D encoder used in the static model.

The third objective is to extend the NeCT forward model to continuous rotation scanning, in which the detector shutter remains open throughout the acquisition and each recorded projection integrates the sample over an angular interval rather than capturing it at a single angle. This mode of operation can increase the information collected per unit time relative to step-and-shoot scanning, at the cost of producing geometrically blurred projections. The thesis derives the modified Beer–Lambert forward model that accounts for this integration and evaluates whether it correctly compensates for the blurring and whether it enables the reconstruction of processes that occur within a single inter-projection interval.

1.3 Thesis Structure

Chapter 2 develops the theoretical background needed to understand the methods. It covers the fundamentals of neural networks and the multilayer perceptron, convolutional approaches to CT reconstruction and their limitations, the theory of implicit neural representations and positional encodings, Neural Radiance Fields as the conceptual origin of the INR-based CT approach, and the multiresolution hash encoding that enables fast training. The second half of the background chapter covers X-ray CT from first principles, including Beer–Lambert's law, cone-beam geometry, classical reconstruction algorithms including FDK and OS-SART-TV, and the challenges of 4D-CT and sparse-view acquisition. The chapter concludes with a detailed description of the NeCT algorithm itself, including both the static 3D model and the dynamic QuadCubes model, the training pipeline, and the hybrid golden-section sampling scheme.

Chapter 3 addresses the first research objective. It derives the memory cost of the baseline NeCT configuration, identifies the GPU hardware constraints that follow from it, and presents a systematic sweep of encoder size parameters including the hashmap capacity, number of levels, and features per level. Reconstruction quality and training time are measured jointly across all configurations, and the effect of allocating encoder capacity non-uniformly across the four encoders is also examined.

Chapter 4 addresses the second research objective. It formalises the design space of encoder architectures in terms of the collision rate and dimensional coupling trade-off, then presents experimental results for the SingleCube, QuadCubes, SexCubes, CombinedCubes, and TriCubes architectures.

Chapter 5 addresses the third research objective. It explains the physical difference between step-and-shoot and continuous rotation scanning, derives the K-step averaged forward model, and presents experimental results on both a static validation dataset and a dynamic dataset.

Chapter 6 summarises the contributions and findings of the thesis and identifies

directions for future work.

Chapter 2

Background

2.1 Neural Networks

A neural network is a parameterised function whose internal parameters are adjusted iteratively until the function approximates a desired input-output relationship. The basic computational unit is the neuron, which computes a weighted sum of its inputs and passes the result through a nonlinear activation function. By stacking many neurons into layers and connecting layers sequentially, a network can represent increasingly complex relationships between its inputs and outputs.

The fundamental architecture used throughout this work is the *multilayer perceptron* (MLP), sometimes called a fully-connected or feedforward network. An MLP consists of an input layer, one or more hidden layers, and an output layer. Each layer ℓ applies a learned affine transformation followed by a nonlinear activation function:

$$\mathbf{h}^{(\ell)} = \sigma\Big(W^{(\ell)}\,\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}\Big), \tag{2.1}$$

where $W^{(\ell)}$ is the weight matrix, $\mathbf{b}^{(\ell)}$ is the bias vector of layer ℓ, and σ denotes the activation function applied element-wise. The input to the first layer is the network input $\mathbf{h}^{(0)} = \mathbf{x}$, and the output of the final layer L is the prediction $\hat{y} = \mathbf{h}^{(L)}$.

The composition of L such layers defines a function $f_\theta : \mathbb{R}^n \to \mathbb{R}^m$, where θ collects all learnable parameters $\{W^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^{L}$. By the universal approximation theorem, an MLP with at least one hidden layer and a nonlinear activation can approximate any continuous function on a compact domain to arbitrary precision given sufficient capacity [Cybenko, 1989, Hornik et al., 1989].

In the choice for activation functions sigmoid $\sigma(z) = (1 + e^{-z})^{-1}$ and hyperbolic tangent $\sigma(z) = \tanh(z)$ are the most historically popular, but saturate for large input magnitudes, causing vanishing gradients in deep networks. The rectified linear unit (ReLU), $\sigma(z) = \max(0, z)$, largely solved this problem and became the standard choice for many years [Nair and Hinton, 2010]. A known failure mode of ReLU is

that neurons whose pre-activation is consistently negative receive no gradient and become permanently inactive. The Leaky ReLU addresses this by allowing a small gradient for negative inputs:

$$\sigma(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0, \end{cases} \tag{2.2}$$

where $\alpha \ll 1$ is a small constant. For applications requiring the network to represent signals with significant high-frequency content, periodic activations such as sinusoidal representation networks (SIREN, $\sigma(z) = \sin(z)$) have been proposed [Sitzmann et al., 2020]. These can represent sharp features without any additional input encoding but are sensitive to initialisation and can be difficult to train stably.

Training a network means finding the parameter vector θ that minimises a scalar loss function $\mathcal{L}(\theta)$ measuring the discrepancy between predictions and observations. For regression tasks the two most common choices are the mean squared error (L2 loss) and the mean absolute error (L1 loss):

$$\mathcal{L}_{\mathrm{L2}} = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{y}_i - y_i \right)^2 , \tag{2.3}$$

$$\mathcal{L}_{\mathrm{L1}} = \frac{1}{N} \sum_{i=1}^{N} \left| \hat{y}_i - y_i \right| . \tag{2.4}$$

L1 loss is less sensitive to outliers than L2 and is often preferred when the measurements contain occasional corrupted samples. Parameters are updated by gradient descent: at each step θ is adjusted in the direction of steepest descent of $\mathcal{L}$. The gradient $\nabla_\theta \mathcal{L}$ is computed by backpropagation, which applies the chain rule layer by layer from the output back to the input [Rumelhart et al., 1986]. In practice the gradient is estimated from a random mini-batch of B data points at each step rather than from the full dataset. The Adam optimiser [Kingma and Ba, 2015], which maintains per-parameter running estimates of the first and second gradient moments, is the standard choice for training coordinate networks.

2.2 Convolutional Neural Networks

While the MLP treats every input element independently, images and other spatially structured signals have strong local correlations. Nearby pixels tend to be similar, and the same local pattern such as an edge or a boundary can appear at any position in the image. Convolutional neural networks (CNNs) exploit this structure by replacing the dense weight matrices of an MLP with learned local filters applied uniformly across the spatial extent of the input [LeCun et al., 1998].

A discrete 2D convolution of an input feature map $\mathbf{X} \in \mathbb{R}^{H \times W}$ with a filter $\mathbf{K} \in \mathbb{R}^{k \times k}$ produces an output feature map $\mathbf{Y}$ whose entries are

$$Y_{i,j} = \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} K_{p,q} \, X_{i+p,\, j+q}. \tag{2.5}$$

The same kernel $\mathbf{K}$ is applied at every spatial position (i, j), so the number of learnable parameters depends only on the kernel size k rather than the image dimensions. This weight sharing gives CNNs translation equivariance: if the input is shifted spatially, the output shifts by the same amount. Each output unit depends on only a $k \times k$ neighbourhood of the input, called its receptive field. Stacking multiple convolutional layers increases the effective receptive field of deeper units geometrically, allowing the network to integrate local evidence across progressively larger regions. Pooling layers, spatially downsample feature maps and introduce a degree of translation invariance beyond the equivariance of the convolution itself.

For CT image reconstruction, CNNs are typically applied in an image-to-image fashion: a reconstruction degraded by sparse-view artefacts, obtained for example from a fast analytical reconstruction, is passed through the network, which is trained to output a cleaner image [Jin et al., 2017]. The U-Net architecture [Ronneberger et al., 2015] has become dominant for this task. It consists of a contracting encoder path that progressively halves spatial resolution while doubling the number of feature channels, and a symmetric expanding decoder path that restores the original resolution using transposed convolutions. Skip connections concatenate feature maps from the encoder to the corresponding decoder stage, allowing the network to recover fine spatial detail that would otherwise be lost during downsampling. CNNs have also been applied directly in the sinogram domain, where a network learns to predict missing projections in a sparse-view sinogram before passing the completed sinogram to a conventional reconstruction algorithm [Hu et al., 2021]. Generative adversarial networks, which couple a generator network with an adversarially trained discriminator, have further improved the perceptual quality of CNN-based CT reconstructions [Goodfellow et al., 2014, Guo et al., 2022].

Despite their strong empirical performance on static image reconstruction tasks, CNN-based approaches carry a set of structural limitations. A CNN must be trained on a dataset of paired examples before it can be deployed, so assembling training data for a new experimental configuration is expensive and time-consuming. Once trained, the network is committed to that configuration; applying it to a new one requires retraining. CNNs have no internal model of the measurement process and learn only a statistical mapping from the training distribution. When the test image differs from that distribution, as is common in novel scientific experiments, the network can confidently produce physically incorrect features [Patwari et al., 2023]. CNNs also

operate on discrete pixel or voxel grids, so the spatial resolution is fixed at training time. Representing a continuously evolving three-dimensional object at arbitrary spatial and temporal coordinates is not a natural fit for the CNN framework, and constructing a four-dimensional reconstruction requires either treating each time step independently or processing large four-dimensional volumes, neither of which is computationally attractive at the resolutions used in micro-CT.

2.3 Implicit Neural Representations

An implicit neural representation (INR), sometimes called a coordinate network or neural field, is a neural network trained to represent a signal as a continuous function of its coordinates. Formally, an INR is a parameterised function

$$f_\theta : \mathbb{R}^n \longrightarrow \mathbb{R}^m, \tag{2.6}$$

where θ denotes the network weights, the input is a coordinate vector such as a spatial position or a position-time pair, and the output is the signal value at that coordinate. The signal is implicit in the sense that it is never stored as a discrete grid of samples. Instead, the continuous function f_θ is the representation itself, and individual values are obtained by evaluating the network at any desired coordinate.

This contrasts with conventional discrete representations. A 3D volume stored as a voxel grid of resolution N^3 requires memory proportional to N^3, growing rapidly with resolution: a 1000^3 volume at 32-bit precision consumes roughly 4 GB. An INR stores only its network weights, whose count is independent of the desired output resolution. The same network architecture can be queried at 128^3 or 1024^3 spatial positions without any change to its parameter count, making INRs inherently resolution-agnostic.

An INR is instance-specific: it is optimised from scratch for each individual signal, using only the measurements of that signal as supervision. No external dataset of prior examples is needed. The optimisation proceeds by defining a differentiable forward model that maps the network's predicted field to the expected measurements, then minimising the discrepancy between those predictions and the actual observations. Because the physics of the measurement process is embedded directly in the optimisation loop, the reconstruction is constrained to be consistent with the known forward model. This prevents the generation of features that cannot be explained by the measurements, which is a significant failure mode of trained CNN approaches [Patwari et al., 2023].

A naive INR, one in which an MLP receives raw coordinate inputs, struggles to represent signals with significant high-frequency content such as sharp material

boundaries. This is a consequence of the spectral bias of neural networks: gradient-based training causes MLPs to learn low-frequency components of a target function far faster than high-frequency ones [Rahaman et al., 2019]. Analysed through the neural tangent kernel (NTK), a standard coordinate-based MLP corresponds to a kernel whose eigenvalue spectrum decays rapidly with frequency, which prevents convergence to high-frequency components within a practical number of optimisation steps [Tancik et al., 2020].

The standard remedy is to map the raw input coordinates $\mathbf{x} \in \mathbb{R}^n$ through a fixed positional encoding $\gamma : \mathbb{R}^n \to \mathbb{R}^d$ before passing them to the MLP, where $d \gg n$:

$$f_\theta(\mathbf{x}) = \mathrm{MLP}_\theta(\gamma(\mathbf{x}))\,. \tag{2.7}$$

The sinusoidal positional encoding popularised by NeRF [Mildenhall et al., 2021] maps each coordinate to a sequence of sines and cosines at exponentially increasing frequencies:

$$\gamma(\mathbf{x}) = \Big(\sin(2^0\pi\mathbf{x}),\ \cos(2^0\pi\mathbf{x}),\ \ldots,\ \sin(2^{L-1}\pi\mathbf{x}),\ \cos(2^{L-1}\pi\mathbf{x})\Big)\,, \tag{2.8}$$

where L is the number of frequency octaves. Tancik et al. [2020] showed using NTK theory that this encoding transforms the effective kernel into a stationary one with a tunable bandwidth, directly addressing spectral bias. An alternative is to use random Fourier features $\gamma(\mathbf{x}) = [\sin(\mathbf{Bx}), \cos(\mathbf{Bx})]$, where each row of $\mathbf{B}$ is sampled from a Gaussian $\mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$ with bandwidth σ chosen to match the signal's frequency content.

Both sinusoidal encodings and random Fourier features are fixed: their parameters are set before training and not updated by gradient descent. A more capable alternative is to learn the encoding jointly with the MLP. The multiresolution hash encoding of Müller et al. [2022] pursues this by storing trainable feature vectors in a set of hash tables indexed by the input coordinates at multiple spatial resolutions. This shifts much of the representational work from the MLP to the encoding tables, allowing a smaller and faster MLP to achieve equal or better reconstruction quality. The hash encoding is described in detail in section 2.3.1 following the introduction of NeRF.

2.3.1 Neural Radiance Fields (NeRF)

Neural Radiance Fields (NeRF), introduced by Mildenhall et al. [2021], demonstrated that a complex 3D scene could be fully represented by the weights of a small MLP trained on a set of 2D photographs taken from known camera positions, and that this representation was sufficient to synthesise photorealistic images from arbitrary

new viewpoints. What made NeRF influential beyond its original application was the mathematical structure it established: a differentiable forward model connecting a continuous volumetric field to 2D observations via a ray integral, trained end-to-end by minimising the discrepancy between simulated and measured projections. This same structure, a continuous scalar field recovered through a physics-based differentiable renderer, is directly applicable to tomographic reconstruction.

Scene Representation

NeRF represents a static scene as a continuous 5D function

$$F_\Theta : (\mathbf{x},\, \mathbf{d}) \longrightarrow (\mathbf{c},\, \sigma), \tag{2.9}$$

where $\mathbf{x} = (x, y, z) \in \mathbb{R}^3$ is a spatial location, $\mathbf{d} \in \mathbb{S}^2$ is a unit viewing direction, $\mathbf{c} = (r, g, b)$ is the emitted RGB colour at that location as seen from direction $\mathbf{d}$, and $\sigma \geq 0$ is the volume density at $\mathbf{x}$. The density $\sigma(\mathbf{x})$ represents the differential probability that a ray is absorbed at $\mathbf{x}$ and is treated as a property of the scene geometry alone, predicted from position only to enforce multi-view consistency. The colour $\mathbf{c}$ depends on both position and direction, allowing NeRF to represent view-dependent effects such as specular reflections.

In practice F_Θ is implemented as an MLP. The spatial coordinate $\mathbf{x}$ is first lifted by the sinusoidal positional encoding and passed through eight fully-connected ReLU layers with 256 channels each, producing σ and a 256-dimensional feature vector. The feature vector is then combined with the encoded viewing direction and passed through one further layer to produce $\mathbf{c}$ [Mildenhall et al., 2021]. Geometry is processed before appearance, which stabilises training by preventing the network from compensating for geometric errors through view-dependent colour.

To render the colour at a pixel, a camera ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ is cast from the camera origin $\mathbf{o}$ into the scene. The expected colour $C(\mathbf{r})$ accumulated along the ray between near and far bounds t_n and t_f is given by the classical volume rendering integral [Kajiya and Von Herzen, 1984]:

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\, \sigma(\mathbf{r}(t))\;\, \mathbf{c}(\mathbf{r}(t), \mathbf{d})\; \mathrm{d}t, \tag{2.10}$$

where the accumulated transmittance $T(t) = \exp\Big(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))\mathrm{d}s\Big)$ is the probability that the ray travels from t_n to t without being absorbed. The integral is approximated using stratified sampling: the interval $[t_n, t_f]$ is divided into N equal bins and one sample t_i is drawn uniformly from each [Mildenhall et al., 2021]. The discrete

approximation is

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^{N} T_i \left(1 - e^{-\sigma_i \delta_i}\right) \mathbf{c}_i, \qquad T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right), \tag{2.11}$$

where $\delta_i = t_{i+1} - t_i$ is the distance between adjacent samples. This expression is fully differentiable with respect to σ_i and $\mathbf{c}_i$, and therefore with respect to the MLP parameters Θ.

NeRF is trained by minimising the squared photometric error between the rendered colour $\hat{C}(\mathbf{r})$ and the observed pixel colour $C(\mathbf{r})$ over all training rays. No 3D supervision is required; the only inputs are the 2D images and the known camera poses. NeRF additionally uses a hierarchical sampling strategy in which a coarse network places samples uniformly and a fine network concentrates additional samples in regions of high density [Mildenhall et al., 2021].

The structural parallel with CT reconstruction is close. In both settings a continuous 3D field is observed indirectly through line integrals, and the goal is to recover the field from a finite collection of such measurements. Table 2.1 summarises the correspondences.

Table 2.1: Structural analogy between NeRF and attenuation-contrast CT.

Concept	**NeRF**	**CT**
Recovered field	Volume density $\sigma(\mathbf{x})$	Attenuation coefficient $\mu(\mathbf{x})$
Measurements	Pixel colours $C(\mathbf{r})$	Detector intensities I_j
Forward model	Volume rendering integral	Beer–Lambert law
Ray sampling	Stratified random sampling	Equidistant ray marching
Loss function	L2 photometric error	L1 intensity error
Input encoding	Sinusoidal $\gamma(\mathbf{x})$	Multiresolution hash grid

The key difference between the two forward models is that in NeRF the transmittance is weighted by emitted colour, whereas in monochromatic CT no colour is emitted and the measured intensity follows Beer–Lambert's law directly:

$$I_j = I_0 \exp\left(-\int_{\text{ray}_j} \mu(\mathbf{r}(t))\, \mathrm{d}t\right). \tag{2.12}$$

Setting $\sigma(\mathbf{x}) \equiv \mu(\mathbf{x})$ and dropping the colour output reduces the NeRF forward model to exactly this expression. The CT reconstruction problem can therefore be addressed using the same INR machinery as NeRF, substituting the Beer–Lambert integral for the volume renderer and choosing an encoding suited to the properties of tomographic data.

Multiresolution Hash Encoding

The sinusoidal positional encoding resolves the spectral bias problem but places all of the representational work on the MLP. Every gradient step updates every weight in the network, and convergence is slow regardless of how well the encoding lifts the input coordinates. The multiresolution hash encoding of Müller et al. [2022] addresses this by distributing the work between the MLP and a set of small trainable lookup tables, allowing the MLP itself to be made much smaller. The result is a speedup of several orders of magnitude over sinusoidal-encoded NeRF at equal or better reconstruction quality [Müller et al., 2022].

The encoding maintains L independent levels, each corresponding to a different spatial resolution. At each level ℓ a grid of resolution N_ℓ is defined, where the resolutions grow as a geometric progression between a coarsest resolution $N_{\min}$ and a finest resolution $N_{\max}$:

$$N_\ell = \left\lfloor N_{\min} \cdot b^\ell \right\rfloor, \qquad b = \exp\left(\frac{\ln N_{\max} - \ln N_{\min}}{L-1}\right). \tag{2.13}$$

Each level stores up to T trainable feature vectors of dimension F, arranged in a flat array. The level's grid vertices are mapped into this array in one of two ways depending on the level's resolution. At coarse levels, where the grid has fewer than T vertices, each vertex maps to a unique array entry (a 1:1 mapping, equivalent to a dense grid). At finer levels, where the number of grid vertices exceeds T, the array is treated as a *hash table* and each vertex is mapped to an entry by a spatial hash function $h : \mathbb{Z}^d \to \mathbb{Z}_T$. The total number of trainable encoding parameters is therefore bounded by $T \cdot L \cdot F$, regardless of the finest resolution $N_{\max}$, making the memory cost a controlled hyperparameter rather than a consequence of resolution. Typical values used in NeCT are $L = 16$ levels, $T = 2^{23}$ entries, $F = 2$ features per entry, $N_{\min} = 16$, and a finest resolution calibrated to the voxel size of the CT scan [Friis et al., 2025].

Given an input coordinate $\mathbf{x} \in \mathbb{R}^d$, the encoding proceeds independently at each level ℓ. The coordinate is scaled by that level's grid resolution to identify the enclosing voxel, and the 2^d integer corner vertices of that voxel are determined. Each corner is mapped to an array index via either the 1:1 mapping at coarse levels or the hash function at fine levels, and the corresponding F-dimensional feature vectors are retrieved. These 2^d feature vectors are then linearly interpolated according to the fractional position of $\mathbf{x}$ within its enclosing voxel, using trilinear interpolation in 3D or the 4D analogue in space-time. The interpolated F-dimensional vector is the output of level ℓ. The outputs from all L levels are concatenated to form the full

encoded representation $\mathbf{y} \in \mathbb{R}^{L \cdot F}$, which is then passed to the MLP:

$$\hat{\mu}(\mathbf{x}) = \mathrm{MLP}_{\Phi}(\mathbf{y}(\mathbf{x};\, \theta))\,, \qquad \mathbf{y} = \Big[\mathrm{interp}_0(\mathbf{x};\, \theta_0),\ \mathrm{interp}_1(\mathbf{x};\, \theta_1),\ \ldots,\ \mathrm{interp}_{L-1}(\mathbf{x};\, \theta_{L-1})\Big]\,. \tag{2.14}$$

Crucially, linear interpolation is differentiable, so gradients flow back through the concatenation and interpolation to the feature vectors θ_ℓ at each level, updating only the 2^d entries involved in each forward pass. For a 3D grid this is 8 entries per level per sample rather than every weight in the MLP. This sparse gradient update is the primary reason the encoding converges so much faster than a purely MLP-based representation [Müller et al., 2022].

At fine resolution levels, multiple grid vertices inevitably map to the same hash table entry. Unlike classical hash tables which resolve such collisions through chaining or probing, the multiresolution hash encoding handles them implicitly. When two vertices collide, their gradients accumulate in the same feature vector during training. The gradients from the region with the highest loss dominate the update, so the shared vector is pulled toward representing the most important detail. The MLP then uses context from the other $L - 1$ levels, which encode the same location at different resolutions with different hash functions, to disambiguate colliding entries at query time. In practice collisions are most frequent at the finest levels, where they concentrate representational capacity on regions of high spatial detail. No explicit data-structure updates such as pruning or splitting are needed at any point during training [Müller et al., 2022], which maps well to GPU parallelism.

Because hash table lookups are $\mathcal{O}(1)$ with no control-flow branching, all L levels can be evaluated in parallel on a GPU. The small MLP that follows requires far fewer floating-point operations than the large MLP needed by sinusoidal encodings. Together these properties enable training to convergence in seconds rather than hours on a single GPU, a speedup of roughly two to three orders of magnitude over the original NeRF implementation [Müller et al., 2022].

The first application of multiresolution hash encoding to CT reconstruction was Neural Attenuation Fields (NAF) by Zha et al. [2022]. NAF parameterises the 3D attenuation coefficient field $\mu(\mathbf{x})$ as a hash-encoded MLP and trains the network by minimising the discrepancy between simulated and measured cone-beam projections using Beer–Lambert's law as the forward model, with no external training data required. NAF showed that the hash encoding's advantages in NeRF, faster convergence and better recovery of high-frequency detail, transfer directly to CT and outperform both sinusoidal-encoded INRs and iterative algorithms at a fraction of the computational cost [Zha et al., 2022]. The physical motivation is that CT attenuation fields are typically smooth within homogeneous material phases but sharp at phase boundaries. Coarse hash levels capture the smooth large-scale variation efficiently

while fine levels concentrate capacity at the boundaries where the self-disambiguation mechanism is most useful.

2.4 X-ray Computed Tomography

2.4.1 Physical Principles

X-ray computed tomography reconstructs the three-dimensional internal structure of an object from a set of its two-dimensional projections. The physical quantity being reconstructed is the linear attenuation coefficient $\mu(\mathbf{r})$, a material-dependent scalar field defined at every point $\mathbf{r} = (x, y, z)$ in the object that describes how strongly the local material absorbs or scatters X-ray photons. Its SI unit is m^{-1} and it depends on both the material density and the photon energy. In attenuation-contrast CT the contrast between different materials arises entirely from differences in μ: dense materials such as rock matrix and precipitated salt have high μ, air-filled pores have low μ, and brine lies in between.

When a monochromatic X-ray beam of initial intensity I_0 travels through a heterogeneous material along a straight ray path, the transmitted intensity I is governed by Beer–Lambert's law:

$$I = I_0 \exp\Big(- \int_{\text{ray}} \mu(\mathbf{r}(t))\, \mathrm{d}t \Big), \tag{2.15}$$

where the integral is taken along the parameterised ray path $\mathbf{r}(t) = \mathbf{o} + t\,\mathbf{d}$, with $\mathbf{o}$ the source position and $\mathbf{d}$ a unit direction vector. Taking the negative logarithm of the transmitted-to-incident intensity ratio gives the line integral of μ along the ray, also called the projection value p:

$$p = -\ln\Big(\frac{I}{I_0}\Big) = \int_{\text{ray}} \mu(\mathbf{r}(t))\, \mathrm{d}t. \tag{2.16}$$

CT reconstruction is the inverse problem of recovering $\mu(\mathbf{r})$ from a finite set of such projection values, each corresponding to a different ray direction and detector pixel. Beer–Lambert's law assumes the X-ray beam is effectively monochromatic and that scattering is negligible; both are standard assumptions in laboratory micro-CT analysis [Kak and Slaney, 2001].

In cone-beam CT (CBCT), a point X-ray source illuminates the entire sample simultaneously and the transmitted intensities are recorded on a flat 2D area detector. This is the geometry of standard laboratory micro-CT instruments. A single projection, also called a radiograph, is the 2D image $\{I_j\}$ recorded by the detector at one angular position of the rotating sample, where j indexes the detector pixels.

The source-to-detector distance and source-to-object distance together determine the geometric magnification and hence the effective voxel size of the reconstruction. As the sample rotates, successive projections sweep a full 360ř arc, and the complete set of projections forms the sinogram, a 3D data volume whose dimensions are the number of angles, and the two detector pixel dimensions. Each row of a 2D sinogram slice traces a sinusoidal curve as the angular position varies, which is the origin of the name.

In practice the continuous Beer–Lambert integral is approximated by a discrete sum over K sample points along the ray:

$$p_j \approx \sum_{k=1}^{K} \mu(\mathbf{r}_k)\, \Delta t, \tag{2.17}$$

where Δt is the sampling step size. CT reconstruction then amounts to inverting the system of equations $\{p_j = \mathcal{A}\mu\}$, where $\mathcal{A}$ is the forward projection operator that maps the attenuation field to the measured projection values. The system is highly underdetermined when the number of projections is small relative to the number of voxels, making the inverse problem ill-posed and requiring regularisation or prior information for a stable solution.

2.4.2 Classical Reconstruction Algorithms

A large body of reconstruction algorithms has been developed to invert the forward projection operator $\mathcal{A}$ from eq. (2.17). They fall into two broad families. Analytical methods derive a closed-form inversion formula from the properties of the Radon transform, and iterative methods formulate reconstruction as an optimisation problem and solve it by successive approximation. The three algorithms used as baselines in NeCT, FDK, OS-SART-TV, and filtered back-projection, span both families and are described here.

Filtered Back-Projection and FDK

The analytical foundation of most classical CT reconstruction is the filtered back-projection (FBP) algorithm, which follows directly from the Fourier slice theorem. The 1D Fourier transform of a parallel-beam projection at angle ϕ is a slice through the 2D Fourier transform of $\mu(\mathbf{r})$ at the same angle. A full set of projections covering 180ř therefore provides complete coverage of the 2D Fourier domain, and μ can be recovered by an inverse 2D Fourier transform. In the spatial domain this is equivalent to back-projecting each filtered projection onto a reconstruction grid. The ramp filter applied before back-projection compensates for the non-uniform polar sampling of the Fourier domain, which would otherwise over-represent low spatial frequencies

and produce blurred reconstructions [Kak and Slaney, 2001].

The Feldkamp–Davis–Kress (FDK) algorithm [Feldkamp et al., 1984] is the cone-beam extension of FBP. It approximates the full 3D cone-beam geometry by treating each detector row as an independent tilted fan-beam slice, applies a cosine-weighted ramp filter to each row of each projection, and back-projects the filtered data into a 3D reconstruction volume. For small cone angles this approximation is accurate, and FDK is accepted as the standard workhorse for laboratory CBCT because it is non-iterative and extremely fast [Kak and Slaney, 2001].

FDK's critical requirement is that the number of projections N_ϕ must satisfy the Shannon–Nyquist criterion, which for a reconstruction of N pixels across the field of view demands approximately $N_\phi \geq \pi N/2$ projections to avoid aliasing artefacts [Kak and Slaney, 2001]. When this condition is met, FDK reconstructions are of high quality. When it is violated, as it inevitably is in fast or sparse-view CT, the reconstructions suffer from characteristic streak artefacts that radiate from high-contrast features and obscure fine pore-level detail.

Iterative methods formulate reconstruction as the minimisation of a data fidelity term measuring the discrepancy between the forward-projected estimate and the measured projections, optionally supplemented by a regularisation term. The basic algebraic reconstruction technique (ART) updates the estimate ray by ray [Gordon et al., 1970]. The simultaneous algebraic reconstruction technique (SART) [Andersen and Kak, 1984] improves convergence by updating all voxels simultaneously using rays from one projection angle at each step. Ordered subsets SART (OS-SART) further accelerates convergence by processing projections in angularly diverse subsets, so each subset update provides approximately as much new information as a full SART iteration [Du et al., 2017].

Adding a total variation (TV) regularisation term to the OS-SART objective yields OS-SART-TV, which is one of the main baselines used for comparison in the NeCT paper. TV regularisation penalises the sum of the magnitudes of the image gradient across all voxels:

$$\mathcal{R}_{\mathrm{TV}}(\mu) = \sum_{\mathbf{r}} \|\nabla\mu(\mathbf{r})\|_2 \,, \tag{2.18}$$

which promotes piecewise-constant solutions, images with flat homogeneous regions separated by sharp boundaries. This is physically motivated for porous media where the attenuation coefficient is approximately uniform within each material phase. TV regularisation can substantially suppress streak artefacts compared to FDK in sparse-view conditions, but it introduces contrast reduction: the penalty discourages gradients everywhere, so small features and fine pore structures tend to be smoothed out and the reconstructed values within homogeneous regions are pulled toward a

common mean [Kak and Slaney, 2001].

Both FDK and OS-SART-TV were designed for fully-sampled static 3D CT, and their limitations become severe in sparse-view and 4D settings. The most direct approach to 4D-CT is to acquire a sequence of independent full 3D scans, reconstructing each with FDK. The temporal resolution is then equal to the time required to complete one full rotation, typically around one hour for laboratory micro-CT, making it impossible to observe fast events that unfold on timescales of seconds to minutes. Reducing the number of projections per time step improves temporal resolution but causes severe streak artefacts in FDK and contrast loss in OS-SART-TV. Neither algorithm can make use of information from other time steps to inform a given frame because each reconstruction is treated independently. Both methods also produce a discrete sequence of independent volumetric images with no model of the object's temporal evolution, so no information is shared across frames.

2.4.3 4D-CT and Dynamic Imaging

4D-CT denotes the acquisition and reconstruction of CT data from a sample that evolves over time, producing a time-resolved sequence of 3D images. The fourth dimension is time, and the full dataset can be thought of as a continuous field $\mu(\mathbf{r}, t)$ sampled at a finite set of spatial voxels and time steps. In geosciences, 4D-CT is applied to study mechanical deformation of rocks, multiphase fluid flow in porous media, CO_2 mineralisation under reservoir conditions, and reactive dissolution of mineral grains [Withers et al., 2021]. The central challenge in all these applications is achieving sufficient temporal resolution to capture the process of interest while maintaining the spatial resolution needed to observe pore-scale phenomena.

The temporal resolution of a 4D-CT scan is ultimately limited by the time required to collect enough angular projections to reconstruct a 3D volume. In conventional sequential scanning a full 360ř rotation is completed before moving to the next time step, giving a temporal resolution on the order of one hour for laboratory micro-CT. Reducing this requires acquiring far fewer projections per time step. The resulting trade-off is that temporal resolution improves as the number of projections per frame decreases, but reconstruction quality degrades as the inverse problem becomes increasingly underdetermined. Several approaches have been developed to mitigate this. One class of methods incorporates prior information into the reconstruction, for example using a high-quality static scan as a reference frame [Chen et al., 2008, Myers et al., 2011]. Another class uses regularisation tailored to the expected temporal evolution, such as penalising the temporal derivative of μ to enforce smooth changes over time [Goethals et al., 2022]. A third class represents the object as a continuous function of space and time, fitting a single model to all projections from the entire

experiment.

The photon flux of the X-ray source is a fundamental constraint on temporal resolution. Synchrotron radiation sources of the third and fourth generation provide a beam brilliance 10–14 orders of magnitude higher than laboratory X-ray tubes [Withers et al., 2021], enabling sub-second exposures and hundreds of projections per second. Sub-second temporal resolution is therefore achievable at synchrotrons, where 4D micro-CT has been applied to phenomena such as drainage dynamics and insect flight mechanics. Standard laboratory micro-CT instruments are far more accessible but each radiograph requires a finite exposure time to achieve an acceptable signal-to-noise ratio, which limits acquisition speed.

The choice of angular sampling scheme significantly affects how much complementary information is gathered per projection in a sparse-view acquisition. In conventional equiangular scanning, projections are collected at equally spaced angles around 360ř. When a full rotation is completed before beginning the next, this is well-suited to static reconstruction but produces temporal aliasing in dynamic experiments: consecutive projections are taken at similar angles, providing redundant information about the current state of the sample.

The golden-angle sampling scheme addresses this by setting the angular increment between successive projections to $\Delta\phi \approx 137.51$ř, which is the angle that most uniformly distributes projections on a circle as more are accumulated [Kohler, 2004]. At any point during the acquisition the projections so far are nearly optimally spread in angle, so a reconstruction from the first N projections is well-conditioned regardless of the total number eventually collected. This property is useful for time-resolved imaging where the effective number of projections per time step is small.

A practical limitation of the pure golden-angle scheme is that consecutive projections are separated by 137.51ř, requiring the sample stage to travel through a large angle between exposures. In laboratory instruments this large step is time-consuming and can cause vibration. The hybrid golden-section scheme used in NeCT experiments [Friis et al., 2025] addresses this: within each full 360ř revolution a fixed number of equidistant projections are collected, but the starting angle of each revolution is offset from the previous one by the golden angle. This preserves rapid continuous rotation within each revolution while ensuring that successive revolutions are angularly offset, so that projections from different time points together cover the angular space as uniformly as possible.

2.5 NeCT: Neural Computed Tomography

NeCT (Neural Computed Tomography) [Friis et al., 2025] is, to the best of the authors' knowledge, the first fully INR-based algorithm for holistic time-resolved

4D-CT reconstruction. It unifies the physical forward model of cone-beam CT with the representational power of hash-encoded implicit neural representations, treating the entire dynamic experiment, all projections from all time steps, as a single joint optimisation problem. The output is a continuous function $\hat{\mu}(\mathbf{r}, t)$, the estimated attenuation coefficient at any spatial coordinate $\mathbf{r}$ and any time t within the duration of the scan, from which volumetric images at arbitrary time points can be decoded by a forward pass through the network.

NeCT consists of two architecturally related but distinct models: a static 3D model for sparse-view single-frame CT reconstruction (an improved version of NAF [Zha et al., 2022]), and a dynamic 4D model called *NeCT QuadCubes* for time-resolved reconstruction. Both share the same training loop and loss function; they differ in the structure of the hash encoding and whether the time coordinate t is included as an input. The static model serves as a benchmark for spatial reconstruction quality, while the dynamic model is the primary contribution relevant to the extensions developed in this thesis.

2.5.1 NeCT Pipeline Overview

The NeCT pipeline operates as a differentiable physics simulation. It takes the measured detector images as supervision and adjusts the network weights until the network's predictions are consistent with every recorded projection. The same four-step loop repeats for every training batch.

At the start of each step, a batch of B rays is sampled from the full set of recorded projections. Each ray $\mathbf{r}_j(t) = \mathbf{o}_j + t\,\mathbf{d}_j$ is defined by its source position $\mathbf{o}_j$ and unit direction $\mathbf{d}_j$, determined by the cone-beam geometry of the instrument and the angular position of the projection from which the ray originates. In the dynamic model, each ray also carries a timestamp t_j corresponding to the acquisition time of its projection. The measured intensity I_j at the corresponding detector pixel is retrieved as the supervision target.

Each ray is then marched through the object volume by sampling K equidistant points $\{\mathbf{r}_k\}_{k=1}^{K}$ along the ray within the sample bounding volume. The number of sample points per ray is not fixed; NeCT uses an adaptive scheme in which K increases linearly from $K_{\text{init}} = 100$ to $K_{\max} = 1.5 \times \max(N_{\text{detector}})$ over the first 30% of training, where $\max(N_{\text{detector}})$ is the number of pixels along the longest detector axis [Friis et al., 2025]. Starting with fewer points reduces computational cost in the early stages when the network is still poorly initialised, and progressively increasing the count ensures the final reconstruction is sampled finely enough to resolve sub-voxel features.

Each sampled point $(\mathbf{r}_k, t_j)$ is passed through the neural network to obtain the

predicted attenuation coefficient $\hat{\mu}(\mathbf{r}_k, t_j)$. In the static model t_j is omitted. The predicted detector intensity for ray j is then computed by discretising Beer–Lambert's law using the midpoint rule:

$$\hat{I}_j = I_0 \exp\left(- \sum_{k=1}^{K} \hat{\mu}(\mathbf{r}_k,\, t_j)\, \Delta s \right), \tag{2.19}$$

where Δs is the step size between adjacent sample points. The exponential is computed from the summed attenuation rather than as a product of per-step transmittances, which is numerically equivalent under the midpoint approximation and more efficient to evaluate.

The predicted intensities $\hat{I}_j$ are compared to the measured intensities I_j using the L1 loss over the batch:

$$\mathcal{L} = \frac{1}{B} \sum_{j=1}^{B} \left| \hat{I}_j - I_j \right|. \tag{2.20}$$

Gradients of $\mathcal{L}$ with respect to all network parameters (Φ, θ), the MLP weights Φ and the hash table feature vectors θ, are computed by automatic differentiation and used to update the parameters via the Adam optimiser [Kingma and Ba, 2015]. The learning rate follows a schedule with a linear warm-up over the first 5 000 batches, after which it decays according to a cosine schedule to 1% of its peak value [Friis et al., 2025]. Base learning rates are 1×10^{-3} for static reconstructions and 2×10^{-4} for dynamic reconstructions. The batch size is 5×10^6 ray sample points per step.

This loop repeats for all training epochs. Because the hash table entries are updated only for the small subset of grid vertices intersected by each batch of rays, just $2^3 = 8$ entries per level per ray for the 3D encoder, the effective gradient update is extremely sparse, which is what gives the hash encoding its computational efficiency.

2.5.2 Static 3D Reconstruction: Improved NAF

The static NeCT model represents the attenuation coefficient as a continuous function of spatial coordinates only, $\hat{\mu} : \mathbb{R}^3 \to \mathbb{R}$, implemented as a hash-encoded MLP. It is architecturally derived from NAF [Zha et al., 2022] but with a refined encoder configuration and training protocol, and was benchmarked against FDK, OS-SART-TV, and NAF on eleven standard 3D CT datasets from the Open SciVis Datasets project [Klacansky, 2017].

The input spatial coordinate $\mathbf{r} = (x, y, z)$ is passed through a single 3D multiresolution hash encoder. The encoder uses a hashmap with $T = 2^{23}$ entries, $L = 16$ resolution levels, a base resolution of $N_{\min} = 16$, and $F = 4$ features per entry, yielding a 64-dimensional concatenated feature vector [Friis et al., 2025]. The MLP that follows has 4 hidden layers each with 128 neurons and Leaky ReLU activations,

with a single linear output neuron producing the scalar $\hat{\mu}(\mathbf{r})$. The total number of trainable parameters is dominated by the hash table, roughly 537 million encoding parameters, compared to a few tens of thousands of MLP weights. In practice only the entries intersected by training rays are ever updated, so the effective active parameter count is much lower. The finest grid resolution $N_{\max}$ is calibrated to the detector resolution of the specific CT scan so that the finest hash grid voxels correspond approximately to the physical voxel size of the reconstruction.

Reconstructions were produced from 49 projections for each dataset, far below the Shannon–Nyquist requirement for the object sizes involved. NeCT was evaluated against FDK, OS-SART-TV, and NAF using peak signal-to-noise ratio (PSNR) and the structural similarity index measure (SSIM) [Wang et al., 2004]. PSNR is defined in terms of the mean squared error (MSE) between the reconstructed image $\hat{\mu}$ and the ground truth μ^*, normalised by the dynamic range R:

$$\mathrm{PSNR} = 10 \log_{10}\left(\frac{R^2}{\mathrm{MSE}(\hat{\mu},\, \mu^*)}\right), \tag{2.21}$$

measured in decibels. SSIM captures perceptual similarity by comparing local statistics of luminance, contrast, and structure; it ranges from 0 to 1 [Wang et al., 2004]. Both metrics are computed volumetrically on the full 3D reconstruction.

NeCT outperformed all three competing methods on both metrics across all eleven datasets [Friis et al., 2025]. On the Stagbeetle dataset, representative of a high-complexity biological specimen, NeCT achieved a PSNR of 45.6 dB and SSIM of 0.996, compared to 44.3 dB / 0.994 for NAF, 41.9 dB / 0.990 for OS-SART-TV, and 28.6 dB / 0.577 for FDK. The improvement over NAF reflects the refined encoder configuration and the extended training protocol. Visually, NeCT reconstructions are free of the streak artefacts that dominate FDK results and the contrast-reduced appearance of OS-SART-TV, showing sharp material boundaries with minimal spurious features. This clean appearance despite using a fraction of the projections required by FDK is attributed in part to the spectral regularisation effect of the hash-encoded INR, which is spectrally biased toward smooth solutions and suppresses high-frequency streak patterns while preserving sharp boundaries between material phases [Friis et al., 2025].

2.5.3 Dynamic 4D Reconstruction: NeCT QuadCubes

The dynamic NeCT model, named QuadCubes, extends the 3D INR to four dimensions by representing the attenuation field as a continuous function of both space and time, $\hat{\mu} : \mathbb{R}^3 \times \mathbb{R} \to \mathbb{R}$. The central design challenge is how to structure the hash encoding for a 4D input space.

The most straightforward extension would be a single 4D hash encoder indexing its tables with (x, y, z, t) corner vertices. This has two problems. A 4D voxel has $2^4 = 16$ corner vertices rather than $2^3 = 8$, doubling the number of table lookups and gradient updates per sample. More severely, the hash collision rate increases dramatically: for a fixed table size T, the expected number of 4D grid vertices per table entry at the finest level grows as $N_{\max}^4/T$, which at the resolutions required for micro-CT becomes large enough that the network's collision disambiguation mechanism is overwhelmed. At the other extreme, one could use many low-dimensional encoders, for example six 2D encoders covering all coordinate pairs, as in the K-Planes [Fridovich-Keil et al., 2023] and HexPlane [Cao and Johnson, 2023] architectures. This avoids the collision problem but each encoder captures only pairwise dependencies, placing a heavy burden on the MLP to integrate information across all four dimensions.

QuadCubes is a compromise between these extremes [Friis et al., 2025]. It uses four independent 3D multiresolution hash encoders, each covering a different triplet of the four space-time dimensions:

$$\text{Encoder 1: } (x, y, z), \quad \text{Encoder 2: } (x, y, t), \quad \text{Encoder 3: } (x, z, t), \quad \text{Encoder 4: } (y, z, t). \tag{2.22}$$

This set covers all four possible 3-element subsets of $\{x, y, z, t\}$, so every pair of coordinates appears together in at least two encoders. Each encoder independently produces a feature vector of dimension $L \cdot F$, and the four vectors are concatenated to form the full MLP input:

$$\hat{\mu}(\mathbf{r}, t) = \mathrm{MLP}_\Phi\Big(\big[\gamma_{xyz}(\mathbf{r};\, \theta_1),\ \gamma_{xyt}(x, y, t;\, \theta_2),\ \gamma_{xzt}(x, z, t;\, \theta_3),\ \gamma_{yzt}(y, z, t;\, \theta_4)\big]\Big), \tag{2.23}$$

where $\gamma_{(\cdot)}$ denotes the trilinear-interpolated hash encoding in the indicated coordinate triplet and θ_i are the trainable feature vectors of encoder i. The MLP has the same architecture as the static model: 4 hidden layers of 128 neurons with Leaky ReLU activations.

Each encoder operates in a well-conditioned 3D hash space where the collision rate remains manageable at micro-CT resolutions. Each encoder also jointly represents three coordinates, so cross-dimensional interactions within each triplet are captured directly in the feature space rather than delegated entirely to the MLP. The overlap between encoders, where the spatial encoder `xyz` shares each of its three dimensions with one of the mixed encoders, provides redundant representations of spatial structure that assist collision disambiguation and give the MLP multiple views of the same location at different times.

A defining property of the QuadCubes model is that it is trained on all projections from the full dynamic experiment simultaneously rather than reconstructing each

time step independently. All N_ϕ projections collected over the entire experiment contribute gradients to the same set of network parameters at every training step. The network learns to distinguish between time-invariant structural features of the rock matrix, which are supported by projections from all angles and all times, and dynamic features such as advancing fluid fronts or dissolving grains, which are supported only by projections in their temporal vicinity. Static structures therefore converge to a sharp well-defined representation, while dynamic features converge more slowly with spatial precision proportional to the rate at which they evolve relative to the projection acquisition rate. The temporal resolution of QuadCubes is consequently scene-dependent: for large, rapidly moving features a temporal resolution approaching the time between two successive projections is achievable [Friis et al., 2025].

Simulations using synthetic 4D datasets of liquid invasion in porous media generated with PoreSpy [Gostick et al., 2019] showed that QuadCubes can resolve dynamics lasting as few as two projection intervals for large features, and approaches full resolution for events spanning ten or more projections. In the experimental Bentheimer sandstone dataset, individual pore-filling events with a characteristic filling time of approximately 10 s and salt dissolution events lasting approximately 30 s were directly observed and quantified [Friis et al., 2025].

2.5.4 Hybrid Golden-Section Sampling

The hybrid golden-section sampling scheme was introduced in the context of 4D-CT acquisition in section 2.4.3. From the perspective of the reconstruction algorithm it is worth examining more closely, because the sampling strategy and the QuadCubes model are designed to work together.

The scheme provides what can be called progressive angular completeness: at any point in the experiment, the projections collected so far are as uniformly distributed in angle as possible. This ensures that the spatial encoder `xyz` receives gradient updates from many different projection directions simultaneously rather than from a narrow angular range, which would leave the spatial reconstruction poorly conditioned in early training. Without angular diversity, the model would converge slowly to an accurate representation of the static rock structure and temporal features would likewise be poorly resolved.

For the temporal encoders `xyt`, `xzt`, and `yzt`, the golden offset between successive revolutions means that each new revolution contributes projections at angles that are maximally distinct from those of all previous revolutions. When a gradient update is computed for a ray from revolution k, the temporal encoders are updated at the time t_k of that revolution. Because each revolution samples an angularly offset range,

successive updates to the temporal encoders carry complementary spatial information and together allow the model to distinguish the attenuation field at t_k from the field at neighbouring time steps.

In the Bentheimer sandstone experiment, 1 400 projections were distributed across 14 revolutions of 100 projections each, with the golden offset applied between revolutions and alternating rotation direction to avoid tangling the inlet supply lines [Friis et al., 2025]. The entire 40-minute imbibition experiment was represented by a single QuadCubes model, decoded at arbitrary times to produce a continuous 3D movie of the brine invasion process.

Chapter 3

Encoder Size and Performance

The original NeCT paper [Friis et al., 2025] treated the encoder configuration as a fixed design choice and evaluated the model by training it to convergence with the largest practical configuration, measuring reconstruction quality as the sole outcome. This approach is appropriate when establishing that the method works at all, but it leaves open the question of whether the chosen configuration is a good one in a broader sense. In practice, the memory footprint of the QuadCubes model with the NeCT configuration is large enough to exclude all but the most expensive research-grade GPUs, and the training time for a single dynamic reconstruction is on the order of tens of hours. Both of these properties are significant barriers to the method's practical use. This chapter investigates whether smaller encoder configurations can achieve competitive reconstruction quality in substantially less time and with substantially less memory, and whether the relationship between encoder capacity and reconstruction quality is smooth enough to permit principled trade-offs.

3.1 Experimental Setup

The Baseline Configuration and Its Cost

The encoder configuration used in the NeCT experiments is defined by the following hyperparameters, expressed in the notation of the `tiny-cuda-nn` hash grid implementation:

The $\log_2$ hashmap size of 23 sets the hash table capacity per level to $T = 2^{23} = 8\,388\,608$ entries. With $L = 23$ levels and $F = 4$ features per entry, the total number of trainable encoding parameters per encoder is $T \times L \times F = 2^{23} \times 23 \times 4 \approx 771$ million. The QuadCubes architecture uses four such encoders, giving a total encoding parameter count of approximately 3.1 billion. Stored at half precision (float16, 2 bytes per parameter), the hash tables alone occupy roughly 6.2 GB of GPU memory.

Table 3.1: Baseline QuadCubes hash encoder configuration from Friis et al. [2025].

Parameter	Value
Encoding type	`HashGrid`
Number of levels (L)	23
Features per level (F)	4
Log_2 hashmap size	23
Base resolution (N_{min})	16
Maximum resolution factor (b)	2

Including the Adam optimiser, which stores a first-moment and a second-moment estimate for every parameter, the optimiser state for the hash tables adds another 12.4 GB. Together with the MLP weights, their optimiser states, intermediate activations, and the projection data loaded on device, the total GPU memory requirement for a QuadCubes training run with the baseline configuration **[TODO: insert measured peak VRAM figure]** exceeds the capacity of consumer and mid-range professional GPUs. In practice this restricts training to high-memory server GPUs such as the Nvidia A100 (80 GB HBM2e), H100 (80 GB HBM3), and H200 (141 GB HBM3e).

The maximum resolution factor $b = 2$ means that each successive level doubles in spatial resolution relative to the previous one. Starting from $N_{\text{min}} = 16$, the finest level has a nominal resolution of $16 \times 2^{22} \approx 67$ million voxels per side. At this resolution the hash table is far smaller than the number of grid vertices, so essentially all fine-level entries are shared by many colliding vertices. The coarsest levels, up to level ℓ where $(16 \times 2^{\ell})^3 \leq T$, use a dense 1:1 mapping; for the baseline configuration this transition occurs at approximately level 7, meaning roughly the top 16 levels are in the hashing regime. Training time for a full dynamic reconstruction with the baseline configuration on a single A100 GPU is **[TODO: insert measured training time]**.

Dataset

All experiments in this chapter use **[TODO: describe the dataset used: PoreSpy simulation / Bentheimer / Open SciVis dataset, resolution, number of projections, acquisition protocol]**. Using a single fixed dataset allows all configurations to be compared under identical conditions. **[TODO: Add any relevant details about how projections were generated or subsampled]**.

Metrics

Reconstruction quality is evaluated using PSNR and SSIM as defined in section 2.5.2. In addition, mean absolute error (MAE) is reported, defined as

$$\mathrm{MAE} = \frac{1}{|\Omega|} \sum_{\mathbf{r} \in \Omega} \left| \hat{\mu}(\mathbf{r}) - \mu^*(\mathbf{r}) \right|, \tag{3.1}$$

where Ω is the set of voxels in the reconstruction volume and μ^* denotes the ground truth attenuation field. MAE is reported alongside PSNR and SSIM because it is in the same physical units as the attenuation coefficient and is therefore directly interpretable in terms of material contrast. All three metrics are computed on the full 3D volume at a fixed decoded time point **[TODO: specify which time point or whether it is averaged]**.

Computational cost is characterised by two quantities: peak GPU memory consumption, measured as the maximum memory allocated during training, and total wall-clock training time from initialisation to convergence. Training is considered converged when **[TODO: define convergence criterion: e.g. validation loss has not decreased by more than X% over Y epochs, or a fixed epoch budget]**. All experiments are run on **[TODO: specify hardware: A100 / H100, memory, number of GPUs]**.

Varied Parameters

The encoder size is controlled by three hyperparameters that dominate the parameter count: the $\log_2$ hashmap size (equivalently, $\log_2 T$), the number of levels L, and the number of features per level F. The MLP width and depth are held fixed at 128 neurons and 4 hidden layers throughout this chapter, since the hash table dominates the parameter count by several orders of magnitude and changes to the MLP have negligible impact on memory and training time relative to changes in the encoder. The effect of MLP size is examined separately in **[TODO: cross-reference if this is covered elsewhere]**.

3.2 Effect of Total Parameter Count

Varying the Hashmap Size

The $\log_2$ hashmap size is the single parameter with the largest effect on both memory and parameter count, since the total number of encoding parameters scales linearly with $T = 2^{\log_2 T}$. Values of **[TODO: list the values swept, e.g. 16, 17, 18, 19, 20, 21, 22, 23]** were evaluated, with L and F held at their baseline values of 23

and 4.

Figure ?? shows PSNR and SSIM as a function of total parameter count for this sweep. **[TODO: Insert figure: x-axis = log-scale parameter count or log2_hashmap_size, y-axis = PSNR / SSIM, one curve each.]**

[TODO: Describe the trend observed in the figure without drawing conclusions: e.g. whether the relationship is approximately linear in log-parameter space, whether there is a plateau, whether there is a minimum size below which quality degrades sharply.]

Varying the Number of Levels

The number of levels L affects both the total parameter count and the range of spatial frequencies that the encoder can represent simultaneously. With $b = 2$ and $N_{\text{min}} = 16$, increasing L extends the encoder's coverage to finer spatial scales, but also adds $T \times F$ new parameters per additional level. Values of **[TODO: list swept values of L]** were evaluated, with the hashmap size and features per level held at their baseline values.

Figure ?? shows reconstruction quality as a function of L. **[TODO: Insert figure and describe observed trend.]**

Varying Features Per Level

The number of features per level F controls the dimensionality of each interpolated feature vector and therefore the amount of information the encoder can store per spatial location at each resolution. Values of **[TODO: list swept values, e.g. 1, 2, 4, 8]** were evaluated. Changing F also changes the input dimension of the MLP, since the MLP receives a concatenated vector of dimension $4 \times L \times F$ for QuadCubes.

Figure ?? shows reconstruction quality as a function of F. **[TODO: Insert figure and describe observed trend.]**

Joint Parameter Sweep

To understand whether total parameter count is a reliable predictor of reconstruction quality regardless of which hyperparameter it comes from, all three sweeps above are plotted together against total encoding parameter count in Figure ??. **[TODO: Insert joint figure. Describe whether the curves from different hyperparameter sweeps coincide or diverge, i.e. whether equal parameter counts from different configurations give equal quality.]**

3.3 Effect on Reconstruction Time

The relationship between encoder size and reconstruction quality is only one dimension of the trade-off. A configuration with slightly lower PSNR but substantially faster training may be preferable for many applications, particularly when the bottleneck is GPU availability or when iterative experimental workflows require multiple reconstructions. The original NeCT approach, which uses the largest feasible configuration and trains for as long as possible, implicitly treats quality as the only objective. Here we instead examine the joint relationship between training time and quality, asking which configurations lie on the Pareto front of quality versus time.

Figure **??** shows PSNR and SSIM as a function of wall-clock training time for all configurations evaluated in section 3.2, with each point representing one configuration at convergence. **[TODO: Insert figure. The plot should resemble a quality-vs-time scatter, ideally with the Pareto front highlighted.]**

[TODO: Describe what the figure shows: e.g. where the bulk of configurations cluster, whether there are configurations that are both faster and more accurate than others, whether the baseline sits on or off the Pareto front.]

Figure **??** shows the same configurations plotted against peak GPU memory consumption. **[TODO: Insert figure. Mark a horizontal line at the memory limits of common GPUs, e.g. 24 GB for RTX 3090/4090, 40 GB for A100 40GB, 80 GB for A100/H100 80GB.]**

[TODO: Describe which configurations fall within the memory budget of consumer or mid-range professional GPUs and whether those configurations achieve competitive reconstruction quality.]

Training Dynamics

Beyond the final converged quality, the rate at which a configuration converges matters for practical use. Figure **??** shows PSNR as a function of training time (rather than epoch) for a representative selection of configurations. **[TODO: Insert figure: multiple curves on a shared time axis, each curve corresponding to one configuration.]**

[TODO: Describe the observed convergence behaviour: e.g. whether smaller configurations converge faster in wall-clock time even if they converge to a lower asymptote, whether larger configurations converge faster per epoch but slower per unit time, whether there is an early phase where all configurations perform similarly.]

3.4 Precision and Accuracy Metrics

PSNR, SSIM, and MAE measure different aspects of reconstruction fidelity and do not always agree in their ranking of configurations. PSNR and MSE are sensitive to large absolute errors at any location, which in a CT reconstruction typically corresponds to errors at high-contrast boundaries. SSIM is sensitive to local structural and contrast differences and tends to reflect the perceptual sharpness of material boundaries. MAE is less influenced by rare large errors and more reflective of the average accuracy across the volume, which includes the large homogeneous interior regions of each material phase where reconstruction is generally easier.

Figure **??** shows PSNR, SSIM, and MAE for all configurations evaluated in this chapter, sorted by total parameter count. **[TODO: Insert figure: three panels or three y-axes, one per metric, configurations on the x-axis.]**

[TODO: Describe whether the three metrics agree in their ranking of configurations, and if not, describe the qualitative nature of the disagreement. For example: do some small configurations achieve high SSIM but low PSNR, suggesting they recover structure but introduce quantitative errors? Do large configurations improve PSNR more than SSIM, suggesting the gains are in isolated high-contrast voxels rather than structural fidelity?]

For the specific application of multiphase flow imaging in porous media, the most physically meaningful metric depends on the intended downstream use. If the reconstruction will be segmented to extract pore-level geometry, SSIM and boundary sharpness are most relevant. If the goal is quantitative phase saturation estimation, MAE in absolute attenuation units is more directly informative. **[TODO: Add any domain-specific discussion once results are available.]**

3.5 Non-Uniform Encoder Sizing

All configurations considered so far assign the same hashmap size, number of levels, and features per level to all four encoders in QuadCubes. This uniform allocation may not be optimal. The spatial encoder `xyz` is responsible for representing the three-dimensional structure of the rock matrix, which is static throughout the experiment and benefits from many observations at many angles. The three mixed encoders `xyt`, `xzt`, and `yzt` are jointly responsible for representing the temporal evolution of the attenuation field, and they receive gradient updates only from projections at the corresponding time point. One might therefore expect that the spatial encoder benefits from higher capacity than each individual temporal encoder, since it accumulates information from all projections while the temporal encoders

share the temporal coverage among themselves.

This section investigates non-uniform configurations in which the spatial encoder `xyz` is assigned a different capacity from the three temporal encoders. The temporal encoders are kept equal to each other throughout, so the configuration space is two-dimensional: the capacity of `xyz` and the capacity of each temporal encoder. Total parameter count is used as a budget, and configurations are evaluated at matched or near-matched total parameter counts to isolate the effect of the allocation from the effect of the total budget.

Spatial-Heavy Allocation

In spatial-heavy configurations, a larger fraction of the total parameter budget is allocated to the `xyz` encoder, with the temporal encoders correspondingly reduced. Table 3.2 lists the specific configurations evaluated. **[TODO: Fill in the table with the actual configurations tested, e.g. varying log2_hashmap_size per encoder type.]**

Table 3.2: Non-uniform encoder configurations evaluated in section 3.5. All configurations use $L =$ **[TODO: value]** levels and $F =$ **[TODO: value]** features per level unless otherwise noted.

Configuration	`xyz` $\log_2 T$	Temporal $\log_2 T$	Total params	Notes
Uniform (baseline)	23	23	**[TODO:]**	Equal allocation
Spatial-heavy A	**[TODO:]**	**[TODO:]**	**[TODO:]**	
Spatial-heavy B	**[TODO:]**	**[TODO:]**	**[TODO:]**	
Temporal-heavy A	**[TODO:]**	**[TODO:]**	**[TODO:]**	
Temporal-heavy B	**[TODO:]**	**[TODO:]**	**[TODO:]**	

Figure **??** shows PSNR and SSIM for all non-uniform configurations compared to the uniform baseline at matched total parameter count. **[TODO: Insert figure.]**

[TODO: Describe the observed effect of allocation on reconstruction quality. Does spatial-heavy allocation improve or degrade quality relative to the uniform baseline? Does the effect depend on the total budget?]

Interpretation

[TODO: Describe any qualitative patterns in where reconstruction errors occur for spatial-heavy versus temporal-heavy configurations, for example whether spatial errors increase or decrease, or whether temporal blurring of fast events changes. Do not draw conclusions; describe observations only.]

Chapter 4

Encoder Architecture

Chapter 3 treated the number and dimensionality of encoders as fixed and varied only their capacity. This chapter takes the opposite approach: the total capacity per encoder is held approximately constant and the structural question of how many encoders to use, and across which coordinate dimensions, is varied instead. The central tension in the design of the hash encoding for a 4D problem is between hash collision rate and dimensional coupling. A single high-dimensional encoder captures all interactions between coordinates directly but suffers from a severe collision problem at any practical memory budget. Many low-dimensional encoders reduce collisions but capture only low-order dependencies between coordinates, placing greater demands on the MLP to integrate the independent streams of information. The QuadCubes architecture of NeCT occupies a particular point in this design space; this chapter evaluates several alternatives systematically.

4.1 Motivation and Design Space

The collision-free hashmap size for a hash encoder of input dimension d can be estimated by requiring the hash table capacity $T = 2^{\log_2 T}$ to be at least as large as the number of grid vertices at the finest resolution level. For a finest resolution $N_{\max}$ in each coordinate direction, the number of vertices is $N_{\max}^d$. Setting $T \geq N_{\max}^d$ gives the minimum collision-free $\log_2$ hashmap size as $d \cdot \log_2 N_{\max}$.

For a representative finest resolution of $N_{\max} = 2^{10} = 1024$ voxels per side, which is a modest value for micro-CT data, this gives:

$$\begin{aligned} d = 2: &\quad \log_2 T_{\text{free}} = 2 \times 10 = 20, \quad T_{\text{free}} \approx 10^6, \\ d = 3: &\quad \log_2 T_{\text{free}} = 3 \times 10 = 30, \quad T_{\text{free}} \approx 10^9, \\ d = 4: &\quad \log_2 T_{\text{free}} = 4 \times 10 = 40, \quad T_{\text{free}} \approx 10^{12}. \end{aligned}$$

At float16 precision with $L = 16$ levels and $F = 4$ features per level, the memory

required for a single collision-free encoder grows from approximately 128 MB for $d = 2$, to roughly 64 GB for $d = 3$, and to approximately 64 TB for $d = 4$. The $d = 4$ case is therefore entirely infeasible with any current hardware, and even the $d = 3$ collision-free encoder would consume more memory than is available on any single GPU. In practice, all configurations operate in the collision regime and rely on the MLP and the multi-level structure to disambiguate collisions, as described in section 2.3.1. The question is how severely collisions degrade reconstruction quality as the input dimension increases, and whether low-dimensional split encoders provide a practical alternative.

The design space explored in this chapter for the dynamic 4D problem is summarised in Table 4.1. Each architecture is characterised by its number of encoders, the input dimension of each encoder, which coordinate subsets are covered, and the practical consequence for memory and collision rate. TriCubes is treated separately in section 4.3 as it addresses the static 3D reconstruction problem rather than the dynamic 4D one.

Table 4.1: Dynamic encoder architecture design space. The collision rate column gives a qualitative assessment relative to a collision-free 3D encoder. Memory estimates assume $L = 16$, $F = 4$, float16, with the hashmap size matched to fit within a fixed total memory budget.

Architecture	Encoders	Dim	Coordinate subsets	Collision regime
SingleCube	1	4	$xyzt$	Extreme
QuadCubes	4	3	All 4 possible 3-subsets	Moderate
SexCubes	6	2	All 6 possible 2-subsets	Low
CombinedCubes	4	3+2+2+2	xyz, xt, yt, zt	Low–Moderate

All architectures use the same MLP: 4 hidden layers of 128 neurons with Leaky ReLU activations. The input dimension of the MLP changes between architectures because the concatenated encoder output dimension is $n_{\text{enc}} \times L \times F$, where n_{enc} is the number of encoders. **[TODO: Confirm whether the MLP width was held fixed across architectures or scaled to match input dimension. If the MLP was widened for SexCubes, state that explicitly here.]**

4.2 QuadCubes (Baseline)

The QuadCubes architecture was described in full in section 2.5.3. It uses four 3D multiresolution hash encoders covering all four possible 3-element subsets of $\{x, y, z, t\}$: `xyz`, `xyt`, `xzt`, and `yzt`. Every pair of coordinates appears together in at least two encoders, and the spatial encoder `xyz` overlaps with each of the three temporal encoders in two dimensions. This overlap gives the MLP multiple

representations of each spatial location and allows the collision disambiguation mechanism to draw on information from encoders with different hash functions.

With the baseline configuration from Chapter 3 ($L = 23$, $\log_2 T = 23$, $F = 4$), the four encoders occupy approximately 6.2 GB of GPU memory at float16. The QuadCubes architecture serves as the primary baseline throughout this chapter. All alternative architectures are evaluated on the same dataset and training protocol, with results normalised or annotated relative to the QuadCubes baseline. **[TODO: State the specific hashmap size used for QuadCubes in this chapter's experiments, noting whether it matches the Chapter 3 baseline or a reduced size chosen for fair comparison.]**

4.3 TriCubes

The architectures evaluated in sections 4.2 and 4.4 to 4.6 all address the dynamic 4D reconstruction problem and include the time coordinate as an input. TriCubes is different in kind: it is an alternative architecture for the static 3D reconstruction problem, in which no temporal encoder is needed and the question is how best to encode the three spatial coordinates (x, y, z).

The static NeCT model, described in section 2.5.2, uses a single 3D multiresolution hash encoder taking the full spatial coordinate as input. The TriCubes architecture replaces this single 3D encoder with three 2D encoders, each covering one of the three possible 2-element subsets of the spatial coordinates: `xy`, `xz`, and `yz`. Their outputs are concatenated and passed to the MLP:

$$\hat{\mu}(\mathbf{r}) = \mathrm{MLP}_{\Phi}\Big(\big[\gamma_{xy}(x, y;\, \theta_1),\ \gamma_{xz}(x, z;\, \theta_2),\ \gamma_{yz}(y, z;\, \theta_3)\big]\Big). \tag{4.1}$$

The motivation is the same as for the 2D encoders in the 4D problem: a 2D encoder with $\log_2 T = 20$ is collision-free at all spatial resolutions up to 1024×1024, whereas a 3D encoder requires $\log_2 T = 30$ to be collision-free at the same per-dimension resolution. The three 2D encoders at $\log_2 T = 20$ together occupy approximately 384 MB, compared to approximately 1.5 GB for a single 3D encoder at $\log_2 T = 23$. The trade-off is the same as in SexCubes relative to QuadCubes: each encoder captures only pairwise spatial dependencies, so the MLP must integrate across all three dimensions from three partially coupled representations rather than from one jointly encoded one.

Figure **??** shows representative reconstruction slices from TriCubes alongside the static NeCT baseline. **[TODO: Insert figure.]**

Figure **??** shows quantitative metrics for TriCubes compared to the static NeCT model. **[TODO: Insert PSNR/SSIM/MAE comparison figure.]**

[TODO: Describe the observed quality difference between TriCubes and the single-encoder static model. Is spatial reconstruction degraded relative to the single 3D encoder, and if so, is the degradation concentrated at features that require coupling across all three spatial dimensions?]

4.4 SexCubes

The SexCubes architecture moves to the other extreme of the design space, using six 2D hash encoders covering all six possible 2-element subsets of $\{x, y, z, t\}$: `xy`, `xz`, `yz`, `xt`, `yt`, and `zt`. This is the approach taken in the K-Planes [Fridovich-Keil et al., 2023] and HexPlane [Cao and Johnson, 2023] architectures for dynamic novel-view synthesis, adapted here to the CT attenuation field.

The central advantage of 2D encoders is a substantially lower collision rate. A 2D encoder with $\log_2 T = 20$ is collision-free at all resolutions up to 1024×1024, as shown in section 4.1. Capping the 2D encoders at $\log_2 T = 20$ therefore ensures that no spatial locality information is lost to collisions, and the per-encoder memory cost is only approximately 128 MB at float16 with $L = 16$ levels and $F = 4$ features. Six such encoders total approximately 768 MB, roughly one eighth the memory of the baseline QuadCubes configuration.

The structural limitation of 2D encoders is that each encoder captures interactions between exactly two coordinates and is entirely blind to interactions involving a third or fourth variable. For example, the `xy` encoder represents the projection of the 4D field onto the xy plane at all times and all z values, with no representation of how the xy distribution varies with z or t. Reconstructing a voxel at (x, y, z, t) from the six plane projections is analogous to recovering a 4D function from its six 2D marginals, which is an underdetermined problem without additional coupling through the network. The MLP must therefore perform a substantially harder integration task than in the 3D encoder architectures, essentially learning to multiply or otherwise combine six weakly informative marginal representations into a coherent 4D field.

Figure **??** shows representative reconstruction slices from SexCubes alongside the QuadCubes baseline. **[TODO: Insert figure showing reconstruction quality comparison. The description from the user is that SexCubes produces similar but worse results than QuadCubes.]**

Figure **??** shows quantitative metrics. **[TODO: Insert PSNR/SSIM/MAE comparison figure.]**

[TODO: Describe the observed reconstruction artefacts or quality differences. For example: are errors distributed uniformly across the volume, or concentrated at features requiring cross-dimensional coupling

such as boundaries that vary along z or temporal transitions? Is the degradation more pronounced in spatial resolution, temporal resolution, or quantitative accuracy?]

The MLP in the SexCubes experiments was **[TODO: specify: same 4-layer 128-neuron MLP as QuadCubes, or widened to compensate for the increased coupling burden]**. **[TODO: If a larger MLP was tested, describe the results here and compare to the standard-MLP SexCubes.]**

4.5 Single Encoder

The SingleCube architecture uses a single 4D multiresolution hash encoder taking (x, y, z, t) as input and mapping directly to a feature vector that is passed to the MLP. This is conceptually the most natural extension of the 3D NAF architecture to 4D: a single encoder represents the full space-time attenuation field without any factorisation.

As shown in section 4.1, a collision-free 4D encoder at a finest resolution of 1024 voxels per side would require a hashmap of $2^{40} \approx 10^{12}$ entries, occupying on the order of 64 TB at float16. Any practically deployable 4D encoder must therefore operate with a hashmap many orders of magnitude smaller than the collision-free threshold, resulting in a severe hash collision load at all but the coarsest levels. With $\log_2 T = 23$ (the same per-level capacity as the QuadCubes encoders), the expected collision rate at the finest level of a 4D encoder is $N_{\max}^4/T$, which for $N_{\max} = 2^{10}$ evaluates to $2^{40}/2^{23} = 2^{17} \approx 131\,000$ vertices per hash table entry. Each stored feature vector must therefore represent an average over more than 100 000 distinct spatial locations simultaneously, and the disambiguation mechanism that relies on multi-level context from other encoders has only the levels of the same encoder to draw on, all of which share the same severe collision regime.

Figure **??** shows representative reconstruction slices from the SingleCube model. **[TODO: Insert figure. The user reports these are extremely noisy.]**

[TODO: Describe the visual character of the SingleCube reconstruction artefacts. For example: are they high-frequency noise distributed across the volume, or lower-frequency structured errors? Are they uniform in space and time or concentrated in particular regions?]

Figure **??** shows quantitative metrics for the SingleCube model compared to QuadCubes. **[TODO: Insert PSNR/SSIM/MAE comparison figure.]**

[TODO: Describe the quantitative metric values. For example: is the PSNR degradation severe enough to indicate that the reconstruction has essentially failed, or is there partial recovery of structure despite the noise?]

The practical conclusion from the SingleCube experiment is that hash collision rate is not merely a theoretical concern but has a direct and observable effect on reconstruction quality. The disambiguation mechanism that allows the QuadCubes encoders to handle moderate collision loads, described in section 2.3.1, depends on information from other encoder levels that were hashed with different spatial offsets. In a single 4D encoder there is no such cross-encoder context available, and the collision load overwhelms the MLP's ability to recover the true attenuation values. **[TODO: Remove or soften this paragraph if you prefer not to draw a conclusion here and instead leave it as an observation.]**

4.6 Combined Cubes Method

The CombinedCubes architecture uses one 3D encoder covering the purely spatial dimensions `xyz` and three 2D encoders covering the space-time pairs `xt`, `yt`, and `zt`. Their outputs are concatenated and passed to the MLP in the same way as QuadCubes:

$$\hat{\mu}(\mathbf{r}, t) = \mathrm{MLP}_{\Phi}\Big(\big[\gamma_{xyz}(\mathbf{r};\, \theta_1),\ \gamma_{xt}(x, t;\, \theta_2),\ \gamma_{yt}(y, t;\, \theta_3),\ \gamma_{zt}(z, t;\, \theta_4)\big]\Big). \tag{4.2}$$

The motivation is to decouple the representation of static spatial structure from temporal dynamics in a way that matches the physical structure of the problem. The spatial encoder `xyz` alone is responsible for representing the time-invariant rock matrix. The three 2D temporal encoders each represent how one spatial coordinate co-varies with time, capturing the projection of the 4D dynamic field onto three orthogonal space-time planes. Together, the four encoders provide both a high-resolution spatial representation and coverage of all space-time relationships, but without requiring any encoder to operate in more than three dimensions.

The key practical consequence of this factorisation is the hashmap size required for the temporal encoders. As established in section 4.1, a 2D encoder is collision-free at $\log_2 T = 20$ for resolutions up to 1024×1024. The temporal encoders in CombinedCubes can therefore be capped at $\log_2 T = 20$ without introducing collisions, regardless of the temporal resolution of the scan. At float16 with $L = 16$ levels and $F = 4$ features per level, each 2D encoder at $\log_2 T = 20$ occupies approximately 128 MB. Three such encoders total approximately 384 MB. The `xyz` encoder, using the same configuration as the QuadCubes spatial encoder at $\log_2 T = 23$, adds a further approximately 1.5 GB. The total hash table memory for CombinedCubes is therefore approximately 1.9 GB, compared to approximately 6.2 GB for the baseline QuadCubes configuration, a reduction of roughly 70%.

Figure **??** shows representative reconstruction slices from CombinedCubes along-

side the QuadCubes baseline. **[TODO: Insert figure. The user reports CombinedCubes gives the same quality as QuadCubes.]**

Figure **??** shows quantitative metrics. **[TODO: Insert PSNR/SSIM/MAE figure.]**

[TODO: Describe the observed quality difference or equivalence between CombinedCubes and QuadCubes. Are the differences within measurement noise, or is there a systematic pattern? For example: is spatial reconstruction identical while temporal sharpness differs slightly, or vice versa?]

Figure **??** shows training time and peak GPU memory consumption for CombinedCubes compared to QuadCubes across **[TODO: the configurations from Chapter 3]**. **[TODO: Insert figure.]**

[TODO: Describe the training time comparison. The user reports CombinedCubes is faster than QuadCubes. Quantify this if possible: how much faster, and does the speedup scale consistently across different encoder sizes?]

The difference in hashmap size between the temporal encoders of CombinedCubes and those of QuadCubes also affects the distribution of hash collisions. In QuadCubes, all four encoders are 3D and all operate at the same $\log_2 T = 23$, meaning all four have a similar collision load at fine resolution levels. In CombinedCubes, the three temporal 2D encoders at $\log_2 T = 20$ are effectively collision-free, so all collision-related disambiguation is handled entirely by the 3D spatial encoder. Whether this concentration of the collision problem into a single encoder is better or worse than distributing it across all four encoders depends on how well the MLP can draw on cross-encoder context, and the answer may depend on the spatial and temporal complexity of the specific dataset. **[TODO: Add observations from the reconstructions if relevant.]**

4.7 Comparative Analysis

Table 4.2 summarises the five architectures across reconstruction quality, memory, and training time. **[TODO: Fill in the table with measured values once results are available.]**

Figure **??** plots all four dynamic architectures on a quality-versus-memory plane and a quality-versus-time plane. **[TODO: Insert figures. Mark the QuadCubes baseline prominently.]**

The design space explored in this chapter can be understood as a trade-off along two axes: collision rate and dimensional coupling. Moving from SingleCube toward SexCubes reduces the collision rate but also reduces the coupling between

Table 4.2: Summary comparison of all encoder architectures on **[TODO: dataset name]**. Memory figures are peak GPU allocation during training. Training time is wall-clock time to convergence. Quality metrics are computed at **[TODO: specify time point or averaged]**.

Architecture	PSNR (dB)	SSIM	MAE	Memory (GB)	Time (h)
SingleCube	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
QuadCubes	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
SexCubes	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
CombinedCubes	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**

coordinates in each encoder, forcing the MLP to perform more of the integration work. QuadCubes and CombinedCubes occupy intermediate positions in this space, but with different compromises: QuadCubes uses four equally capable 3D encoders and accepts moderate collisions across all of them, while CombinedCubes uses one capable 3D encoder for spatial structure and three lightweight but collision-free 2D encoders for temporal dynamics.

[TODO: Describe the overall pattern in the results: which architectures are on the Pareto front of quality versus memory, which are dominated, and whether the CombinedCubes and QuadCubes architectures are distinguishable in practice or effectively equivalent. Do not draw conclusions; describe what the data shows.]

Chapter 5

Continuous Scanning

The acquisition protocols described in Chapters 2 and 3 all assume a step-and-shoot scanning mode, in which the sample is brought to a stop at each angular position before the detector shutter opens, a projection is recorded, and the shutter closes again before the sample rotates to the next position. This is the standard operating mode for laboratory micro-CT instruments and the mode for which the original NeCT forward model was designed. An alternative is continuous rotation scanning, sometimes called fly-scan, in which the sample rotates at a constant angular velocity and the shutter remains open for the full duration of the scan. This chapter describes the physical difference between the two modes, derives the modified forward model required to reconstruct data acquired in continuous rotation mode, and presents experimental results evaluating the implementation within NeCTv2.

5.1 Motivation

In step-and-shoot mode the sample is stationary during each exposure, so every photon reaching the detector during that exposure passed through the sample at the same angular orientation. Each detector pixel records the line integral of μ along a single well-defined ray at a single known angle. The cost of this geometric cleanliness is time. Between each exposure the sample must decelerate to a stop, the controller waits for mechanical vibration to damp out, the shutter opens, the exposure runs, the shutter closes, and the motor accelerates again to the next position. For instruments where each exposure is short relative to the motor settling time, a significant fraction of the total acquisition time is spent stationary but not imaging. This dead time represents information that could in principle have been collected.

In continuous rotation mode the shutter remains open throughout. The sample rotates steadily and the detector integrates photons over the full angular range swept during the exposure time. If the exposure time is τ and the angular velocity is ω, then each projection integrates the sample over an angular interval of $\Delta\phi = \omega\tau$

radians. The detector pixel at position j no longer records a single line integral. It records the superposition of line integrals along all rays through the sample at all angles within $[\phi_s,\ \phi_s + \Delta\phi]$, weighted by the time spent at each angle. The resulting projection is geometrically blurred in the angular direction, and the severity of the blur is proportional to $\Delta\phi$.

For a step-and-shoot scan with the same total acquisition time, the number of angular positions visited is fixed by the motor speed and settling time. Continuous rotation removes the settling overhead and can therefore visit more angular positions in the same wall-clock time, or equivalently sample a finer angular grid with no increase in total acquisition time. For dynamic experiments this has a direct consequence: processes that occur faster than the step-and-shoot inter-frame interval may be partially or fully unresolvable in step-and-shoot mode, but if continuous rotation reduces the effective time between adjacent projections those same processes become accessible. The trade-off is that the projections acquired in continuous mode are individually less geometrically precise, and a reconstruction algorithm that treats them as if they were step-and-shoot projections will see systematic errors in the forward model.

Naively applying the standard Beer–Lambert forward model to continuously acquired data treats each projection as if it were recorded at a single angle, typically the midpoint of the swept interval. For small $\Delta\phi$ this is a reasonable approximation and the resulting reconstruction artefacts are minor. As $\Delta\phi$ increases, the mismatch between the assumed single-angle forward model and the true integrated forward model grows, and the reconstruction begins to exhibit characteristic angular smearing artefacts that are distinct from the streak and ring artefacts of sparse-view or detector-error origin. Correctly accounting for the angular integration in the forward model suppresses these artefacts and recovers the geometric information embedded in the blurred projections.

5.2 Forward Model for Continuous Scanning

In step-and-shoot mode the intensity recorded at detector pixel j during an exposure at angle ϕ is given by Beer–Lambert's law integrated along the single ray $\mathbf{r}_j^{\phi}$ defined by the source position, detector pixel position, and sample orientation at angle ϕ:

$$I_j^{\phi} = I_0 \exp\left(- \int_{\mathbf{r}_j^{\phi}} \mu(\mathbf{r}(t))\, \mathrm{d}t \right). \tag{5.1}$$

In continuous rotation mode the sample rotates from angle ϕ_s to angle $\phi_e = \phi_s + \Delta\phi$ during the exposure. Assuming the angular velocity $\omega = \Delta\phi/\tau$ is constant and the detector response is linear in photon count, the recorded intensity is the time-average

of the instantaneous Beer–Lambert transmission over the exposure interval:

$$I_j^{[\phi_s,\phi_e]} = \frac{1}{\Delta\phi}\int_{\phi_s}^{\phi_e} I_0 \exp\!\left(-\int_{\mathbf{r}_j^{\phi}} \mu(\mathbf{r}(t))\,\mathrm{d}t\right)\mathrm{d}\phi. \tag{5.2}$$

This integral over ϕ has no closed form in general because the ray $\mathbf{r}_j^{\phi}$ rotates with the sample and the attenuation field μ is an arbitrary function. It is approximated by dividing the swept interval $[\phi_s, \phi_e]$ into K equal sub-intervals and evaluating the Beer–Lambert transmission at the midpoint angle of each sub-interval:

$$\hat{I}_j^{[\phi_s,\phi_e]} = \frac{1}{K}\sum_{k=0}^{K-1} I_0 \exp\!\left(-\sum_p \hat{\mu}\big(\mathbf{r}_p^{\phi_k},\, t_j\big)\,\Delta s\right), \qquad \phi_k = \phi_s + \left(k+\tfrac{1}{2}\right)\frac{\Delta\phi}{K}, \tag{5.3}$$

where $\mathbf{r}_p^{\phi_k}$ are the equidistant sample points along the ray at sub-step angle ϕ_k, Δs is the step size, and t_j is the timestamp of the projection. The parameter K is the number of accumulation steps. Setting $K = 1$ recovers the standard single-angle forward model with the ray placed at the midpoint angle $\phi_s + \Delta\phi/2$, which is equivalent to the step-and-shoot forward model with a possible angular error of up to $\Delta\phi/2$. Increasing K improves the accuracy of the approximation at the cost of K times as many network evaluations per ray per training step.

Algorithm 1 gives the pseudocode for the continuous-scan forward pass implemented in NeCTv2.

Algorithm 1 Continuous-Scan Forward Pass in NeCTv2

Require: Angular interval $[\phi_s, \phi_e]$, accumulation steps K, ray index j, timestamp t_j
$\Delta\phi \leftarrow (\phi_e - \phi_s)/K$
$\hat{I}_j \leftarrow 0$
for $k = 0, \ldots, K-1$ **do**
  $\phi_k \leftarrow \phi_s + (k + 0.5)\,\Delta\phi$ ▷ Mid-point of sub-interval k
  Compute ray geometry $\mathbf{r}_j^{\phi_k}$ from source and detector positions at angle ϕ_k
  Sample P equidistant points $\{\mathbf{r}_p^{\phi_k}\}$ along the ray within the sample volume
  $\hat{I}_j \leftarrow \hat{I}_j + \frac{1}{K}\cdot I_0 \exp\!\big(-\sum_{p=1}^{P} \hat{\mu}\big(\mathbf{r}_p^{\phi_k},\, t_j\big)\,\Delta s\big)$
end for
return $\hat{I}_j$

The computational cost of this forward pass scales linearly with K relative to the standard single-angle pass, since each sub-step requires a full ray march through the volume and a full set of network evaluations. The gradient with respect to the network parameters flows back through each of the K Beer–Lambert integrals independently, so the backward pass also scales linearly with K. In practice the total training time for a continuous-scan reconstruction with K accumulation steps is therefore approximately K times the training time for the equivalent step-and-shoot reconstruction, not accounting for any reduction in the number of projections that

continuous scanning may provide.

5.3 Static Dataset Experiments

To validate the continuous-scan forward model in isolation from the additional complexity of dynamic reconstruction, it was first applied to a static dataset. A static reconstruction does not require the temporal encoders of QuadCubes and uses the 3D hash-encoded MLP described in section 2.5.2. The dataset used was **[TODO: describe the static dataset: name, resolution, number of projections, and how the continuous-scan data was simulated or acquired. If the data was simulated, describe how $\Delta\phi$ was set and how the ground truth projections were generated from the reference volume]**.

The purpose of the static experiment is not to demonstrate a benefit from continuous scanning, since a static sample does not change between exposures and the angular blurring is a pure artefact rather than meaningful physical information. Instead the static experiment tests whether the forward model correctly accounts for the blurring: given projections that were synthetically generated with a known $\Delta\phi$, a reconstruction using the correct K-step forward model should recover the same quality as a reconstruction from step-and-shoot projections at the midpoint angles, while a reconstruction using $K = 1$ on the same blurred data should show degraded quality proportional to $\Delta\phi$.

Figure **??** shows representative reconstruction slices for a range of K values at a fixed $\Delta\phi =$ **[TODO: value]**. **[TODO: Insert figure showing reconstructions at $K = 1, 2, 4, 8$ or similar.]**

Figure **??** shows PSNR and SSIM as a function of K for **[TODO: several values of $\Delta\phi$]**. **[TODO: Insert figure.]**

The static results showed that the multi-step accumulation forward model successfully compensates for the angular blurring introduced by continuous scanning. At $K = 1$ the reconstruction exhibits artefacts consistent with the angular mismatch between the assumed midpoint ray and the true integrated projection. As K increases, reconstruction quality improves and converges at **[TODO: describe at which K the quality plateaus and whether the plateau value matches step-and-shoot quality]**. Beyond a certain K the improvement saturates because the midpoint approximation within each sub-interval becomes negligibly small relative to other sources of reconstruction error. The value of K at which saturation occurs depends on $\Delta\phi$: larger swept angles require more sub-steps to represent the integral accurately. **[TODO: Quantify if possible: for example, $K = 4$ was sufficient for $\Delta\phi \leq 2$ř, while $K = 8$ was needed for $\Delta\phi \leq 5$ř.]**

5.4 Dynamic Dataset Experiments

The static experiment established that the continuous-scan forward model is correctly implemented and that the accumulation-step approximation converges at a practical value of K. The dynamic experiment extends this to a time-resolved reconstruction, where continuous scanning provides a physical advantage over step-and-shoot: because the shutter never closes, the detector captures information continuously and processes that occur within a single exposure interval are reflected in the recorded intensity rather than being missed between frames.

The dataset used for the dynamic experiment was **[TODO: describe the dynamic dataset: PoreSpy simulation, Bentheimer, or other. State the angular velocity, exposure time, resulting $\Delta\phi$, total scan duration, and number of projections.]**

[TODO: The dynamic reconstruction was confirmed to work. Describe the reconstruction at a qualitative level here once results are available: for example, whether fast pore-filling events that are blurred or absent in a $K = 1$ reconstruction become sharper or more accurately timed when $K > 1$. Insert representative figures comparing dynamic reconstructions at different K values.]

Figure **??** shows **[TODO: dynamic reconstruction slices at several time points comparing $K = 1$ and $K =$ [value].]**

[TODO: Describe the observed difference between $K = 1$ and higher K values for the dynamic case. For example: are temporal boundaries between phases sharper? Are events that occur within a single inter-projection interval captured, whereas they were missed or smeared in the $K = 1$ reconstruction?]

5.5 Discussion

The continuous-scan forward model extends NeCTv2 to a broader class of CT acquisition protocols without requiring any change to the network architecture or training procedure. The only modification to the pipeline is the replacement of the single-ray Beer–Lambert evaluation with the K-step averaged forward pass of Algorithm 1. The additional computational cost is proportional to K and is therefore predictable and controllable.

The benefit of continuous scanning is most significant for dynamic experiments where the process being imaged has a characteristic timescale comparable to or shorter than the inter-frame interval of a step-and-shoot acquisition. In that regime, step-and-shoot imaging misses events that occur between frames, while continuous

scanning integrates over those events and records their signature in the blurred projection data. Recovering sharp reconstructions from these blurred projections requires the $K > 1$ forward model: using $K = 1$ on continuously acquired data produces reconstructions with angular smearing artefacts, but using the correct K recovers the geometric information embedded in the blurred projections.

The practical cost of continuous scanning relative to step-and-shoot is that individual projections are geometrically less precise. At small $\Delta\phi$ this is negligible. At larger $\Delta\phi$ the projections carry ambiguous angular information and the reconstruction problem becomes more ill-posed, in the sense that a given observed pixel value is consistent with a wider range of attenuation fields. Whether this additional ambiguity is outweighed by the increased angular sampling rate and the elimination of inter-frame dead time depends on the specific sample and process being imaged, and **[TODO: describe any observations from the experimental results that speak to this trade-off]**.

The value of K required for convergence of the accumulation approximation is a practical consideration for users. The static results suggest that **[TODO: restate the K saturation finding from section 5.3]**. Since training time scales linearly with K, choosing the smallest K that provides adequate forward model accuracy is important for keeping reconstruction times manageable.

[TODO: Describe any remaining limitations: e.g. whether the model assumes a fixed angular velocity, whether it handles alternating rotation direction as in the Bentheimer experiment, and whether it extends naturally to the hybrid golden-section scheme used in dynamic NeCT acquisitions.]

Chapter 6

Conclusion

6.1 Summary of Contributions

[TODO: Bullet-point recap of what was achieved in each chapter.]

6.2 Limitations and Future Work

[TODO: Remaining open questions: spatiotemporal resolution quantification, hyperparameter sensitivity, training time, extension to polychromatic sources, parallel-beam synchrotron settings, energy-resolved CT.]

Bibliography

A. H. Andersen and A. C. Kak. Simultaneous algebraic reconstruction technique (SART): A superior implementation of the ART algorithm. *Ultrasonic Imaging*, 6 (1):81–94, 1984. doi: 10.1177/016173468400600107.

Ang Cao and Justin Johnson. HexPlane: A fast representation for dynamic scenes. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 130–141, 2023.

Guang-Hong Chen, Jie Tang, and Shuai Leng. Prior image constrained compressed sensing (PICCS): a method to accurately reconstruct dynamic CT images from highly undersampled projection data sets. *Medical Physics*, 35(2):660–663, 2008. doi: 10.1118/1.2836423.

George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989. doi: 10.1007/BF02551274.

Yang Du, Gao Yu, Xia Xiang, Xiaoming Wang, and Yves De Deene. Convergence of SART + OS + TV iterative reconstruction algorithm for optical CT imaging of gel dosimeters. *Journal of Physics: Conference Series*, 847:012025, 2017. doi: 10.1088/1742-6596/847/1/012025.

L. A. Feldkamp, L. C. Davis, and J. W. Kress. Practical cone-beam algorithm. *Journal of the Optical Society of America A*, 1(6):612–619, 1984. doi: 10.1364/JOSAA.1.000612.

Sara Fridovich-Keil, Giacomo Meanti, Frederik Warburg, Benjamin Recht, and Angjoo Kanazawa. K-Planes: Explicit radiance fields in space, time, and appearance. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12479–12488, 2023.

Henrik Friis, Håkon Nese, Giacomo Luani, Colin Pryme, Lars Rennan, Basab Chattopadhyay, Anders Kristoffersen, Ole Jakob Mengshoel, and Dag Werner Breiby. Implicit neural representation for fast 4D computed tomography of

multiphase flow in porous media. *Communications Physics*, 8:339, 2025. doi: 10.1038/s42005-025-02249-0.

Wout Goethals et al. Dynamic CT reconstruction with improved temporal resolution for scanning of fluid flow in porous media. *Water Resources Research*, 58: e2021WR031365, 2022. doi: 10.1029/2021WR031365.

Ian Goodfellow et al. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, volume 27, pages 2672–2680, 2014.

Richard Gordon, Robert Bender, and Gabor T. Herman. Algebraic reconstruction techniques (ART) for three-dimensional electron microscopy and X-ray photography. *Journal of Theoretical Biology*, 29(3):471–481, 1970. doi: 10.1016/0022-5193(70)90109-8.

Jeff Gostick et al. PoreSpy: A Python toolkit for quantitative analysis of porous media images. *Journal of Open Source Software*, 4(37):1296, 2019. doi: 10.21105/joss.01296.

Ziyu Guo et al. Physics-assisted generative adversarial network for X-ray tomography. *Optics Express*, 30(13):23238–23259, 2022. doi: 10.1364/OE.459948.

Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989. doi: 10.1016/0893-6080(89)90020-8.

Dianlin Hu et al. Hybrid-domain neural network processing for sparse-view CT reconstruction. *IEEE Transactions on Radiation and Plasma Medical Sciences*, 5 (1):88–98, 2021. doi: 10.1109/TRPMS.2020.3011413.

Kyong Hwan Jin, Michael T. McCann, Emmanuel Froustey, and Michael Unser. Deep convolutional neural network for inverse problems in imaging. *IEEE Transactions on Image Processing*, 26(9):4509–4522, 2017. doi: 10.1109/TIP.2017.2713099.

James T. Kajiya and Brian P. Von Herzen. Ray tracing volume densities. In *Proceedings of the 11th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, pages 165–174, 1984. doi: 10.1145/800031.808594.

Avinash C. Kak and Malcolm Slaney. *Principles of Computerized Tomographic Imaging*. Society for Industrial and Applied Mathematics, Philadelphia, 2001.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

Pavol Klacansky. Open SciVis datasets. `https://klacansky.com/open-scivis-datasets/`, 2017.

Thomas Kohler. A projection access scheme for iterative reconstruction based on the golden section. In *IEEE Symposium Conference Record Nuclear Science 2004*, volume 6, pages 3961–3965, 2004. doi: 10.1109/NSSMIC.2004.1462462.

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.

Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021. doi: 10.1145/3503250.

Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics*, 41(4):102:1–102:15, 2022. doi: 10.1145/3528223.3530127.

Glenn R. Myers, Andrew M. Kingston, Trond K. Varslot, Matthew L. Turner, and Adrian P. Sheppard. Dynamic tomography with a priori information. *Applied Optics*, 50(20):3685–3690, 2011. doi: 10.1364/AO.50.003685.

Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814, 2010.

Mayank Patwari et al. Reducing the risk of hallucinations with interpretable deep learning models for low-dose CT denoising: comparative performance analysis. *Physics in Medicine & Biology*, 68(19):19LT01, 2023. doi: 10.1088/1361-6560/acf5d0.

Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred A. Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 5301–5310, 2019.

Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241. Springer, 2015.

David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986. doi: 10.1038/323533a0.

Vincent Sitzmann, Julien N.P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems*, volume 33, pages 7462–7473, 2020.

Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Advances in Neural Information Processing Systems*, volume 33, pages 7537–7547, 2020.

Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004. doi: 10.1109/TIP.2003.819861.

Philip J. Withers et al. X-ray computed tomography. *Nature Reviews Methods Primers*, 1:1–17, 2021. doi: 10.1038/s43586-021-00015-4.

Ruyi Zha, Yanhao Zhang, and Hua Li. NAF: Neural attenuation fields for sparse-view CBCT reconstruction. In *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 442–452. Springer, 2022.

Appendix A

Supplementary Experimental Details

[TODO: Detailed hyperparameter tables, full benchmark results, extra figures.]

Appendix B

NeCTv2 Hyperparameter Reference

[TODO: Complete listing of all hyperparameters used in each experiment.]